

Dissertation on

"Adaptive Study Planning for Self-Directed Learners"

Submitted in partial fulfilment of the requirements for the award of degree of

Master of Technology in
Data Science and Artificial Intelligence

UE20CS972 Project Phase - 2

Submitted by:
Rohit Saji
PES2PGE24DS201

Under the guidance of
Prof. Ramesh Prakash Guledgudd
PES University

May – October 2026

M.TECH IN DATA SCIENCE AND ARTIFICIAL INTELLIGENCE
FACULTY OF ENGINEERING
PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

FACULTY OF ENGINEERING

CERTIFICATE

This is to certify that the dissertation entitled "Adaptive Study Planning for Self-Directed Learners" is a bonafide work carried out by **Rohit Saji (PES2PGE24DS201)** in partial fulfilment for the completion of Project Phase - 2 (UE20CS972) in the Program of Study – Master of Technology in Data Science and Artificial Intelligence under the rules and regulations of PES University, Bengaluru during the period May 2026 – October 2026. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The dissertation has been approved as it satisfies the academic requirements in respect of project work.

External Viva

Name of the Examiners	Signature with Date
1. _____	_____
2. _____	_____

Signature	Signature	Signature
Prof. Ramesh Prakash Guledgudd	Dr. Sandesh B J	Dr. Keshavan B K
PES University	Chairperson	Dean of Faculty

Declaration

I hereby declare that the Project Phase - 2 entitled “Adaptive Study Planning for Self-Directed Learners” has been carried out under the guidance of Prof. Ramesh Prakash Guledgudd, and submitted in partial fulfilment of the course requirements for the award of the degree of Master of Technology in Data Science and Artificial Intelligence of PES University, Bengaluru, during the academic semester May – October 2026.

Acknowledgement

I would like to express my gratitude to my guide for continuous guidance and support throughout this project, and to the project coordinators for their help. I thank the Chairperson and faculty of the Department, PES University, for their support, and the university leadership for the opportunities provided. Finally, I extend heartfelt thanks to my family and friends for their constant encouragement.

Abstract

Self-directed learners routinely commit to a fixed study plan and fall behind or move ahead within days, yet the plan does not adapt and the learner has no reliable signal of how far off-track they are. Phase I of this work built an open-loop adaptive planner that estimates the learner's true study pace from session history, detects behavioral shifts in that pace, and projects the completion date with an uncertainty band, regenerating the plan when the projection diverges from the deadline. The open loop adapts to pace but never checks learning: logged hours do not prove that studying produced understanding. Phase II closes the loop with assessment-based verification of graded evidence. The system generates grounded assessments in three question families from the learner's own materials, grades each family with an authoritative arm, tracks mastery with Bayesian knowledge tracing, guides with a Socratic ladder and a gated reveal, returns advisory feedback copy, runs mixed practice through the library and practice hub, and replans from the loop verdict. Each component is evaluated against baselines and live runs, and the closed loop undergoes integration validation on synthetic longitudinal histories across four learner archetypes, with no fixed-plan control. Learning gain is not measured: the replay covers integration behavior on synthetic histories, and efficacy on learning remains future work. The selected models are realized in a working local-first web application.

Keywords: adaptive study planning, knowledge tracing, grounded generation, rubric grading, learning analytics.

Contents

Definitions and Abbreviations **10**

1 Introduction **11**
1.1 Background . . . 11
1.2 Problem Statement . . . 11
1.3 Objectives . . . 12
1.4 Scope of the Project . . . 13
1.5 Organization of the Report . . . 13

2 Literature Survey **14**
2.1 Knowledge Tracing and Mastery . . . 14
2.2 Automatic Item Generation and KC Tagging . . . 15
2.3 Large Language Model (LLM) and Sandbox Grading . . . 16
2.4 Socratic Guidance and Feedback . . . 17
2.5 Methodology for Sourcing the Surveyed Work . . . 18
2.6 Summary of Approaches . . . 19

3 System Requirements Specification **21**
3.1 Hardware Requirements . . . 21
3.2 Software Requirements . . . 22
3.3 Functional Requirements . . . 22
3.4 Non-Functional Requirements . . . 23
3.5 Product Perspective . . . 24
3.6 Assumptions, Dependencies and Constraints . . . 24
3.7 Operating Environment . . . 25

4 Proposed Methodology **26**
4.1 The Closed Learning Loop . . . 26
4.2 Candidate Models and Selected Approach . . . 26
4.2.1 Item Generation . . . 27
4.2.2 Grading . . . 27
4.2.3 Mastery and Difficulty . . . 27
4.2.4 Guidance and Feedback . . . 27
4.2.5 Loop Verdict . . . 28
4.3 Comparison Methodology . . . 28

4.4 System Design and Evolution 30
4.5 System Architecture 30

5 Implementation Details **32**
5.1 Development Environment 32
5.2 Datasets and Their Sources 32
5.3 Model and Service Implementation 34
5.4 Evaluation Metrics and Reproducibility 36
5.5 The Phase II Application 37
5.5.1 Materials and Ingestion 37
5.5.2 Assessment Generation 38
5.5.3 Taking and Review 38
5.5.4 Practice and Guidance 38
5.5.5 Feedback and the Closed Loop 39

6 Results and Discussion **42**
6.1 Generation Quality 42
6.2 Live Verification 45
6.3 Closed-Loop Decision 45
6.4 Synthetic Longitudinal Validation 45
6.5 Discussion 46
6.6 Threats to Validity 47
6.7 Summary 47

7 Conclusion and Future Work **48**
7.1 Conclusion 48
7.2 Future Work 49
7.3 Limitations 49

A Supplementary Tables **51**
A.1 Adherence Emission 51
A.2 Verdict Instants 51
A.3 Second-Corpus Questions and Endpoint Pointers 52
A.4 Prompts and Reproduction 52

Bibliography **55**

List of Figures

2.1 The assessment-verified closed loop this chapter motivates: generated items are graded per family, grades update mastery, the verdict steers the plan, and the plan points back at the material. Each stage is motivated in Sections 2.1 to 2.4, and the sandbox and feedback-copy stages are implementation-only. 18

4.1 Guide ladder state machine: each step streams against retrieved chunks and only an explicit confirmation opens Reveal. 28
4.2 Loop verdict router: the judge proposes over the week signals and the code gate disposes. 28
4.3 The two-leg comparison. Fixed golds score each candidate arm behind a fail-closed gate; the synthetic replay then validates the assembled loop after the gate fix. 29
4.4 The deployment that realizes Figure 2.1. The client derives views from the local log, the service fits models to history with the judge folded inside, Supabase acts as the data hub, and the sidecars serve on demand. 31

5.1 The material library: ready cards offer practice while the failed card keeps its badge with a retry action. 37
5.2 Assessment generation: one material, one family, and one scoped chapter per question. 38
5.3 Taking and review: the server grade with per-criterion rubric feedback. 39
5.4 Practice run: the lazily resolved question with the Socratic coach anchored to the attempt. 40
5.5 The complete Phase II application flow: materials reach readiness, generation binds one family per question, taking collects answers under server grading, practice resolves lazily with the coach, and the verdict completes the chain. 40
5.6 Feedback and the loop: the inspectable evidence trail with the advisory recommendation. 41

List of Tables

2.1 Summary of approach families, with strengths and limitations for this project. . . . 19

3.1 Hardware environment. . . . 21
3.2 Software environment by tier. . . . 22

4.1 Candidate approaches per loop stage and the approach selected for the delivered system. . . . 29
4.2 Evaluation metrics by loop stage. . . . 30
4.3 Phase II event types: writer, payload, and reader. . . . 31

5.1 Development environment: tools and libraries. The first eleven rows carry the Phase I stack; the last seven add the Phase II loop. . . . 33
5.2 Datasets used in Phase II and their sources. . . . 33
5.3 Main packages and their roles. . . . 35
5.4 Phase II service surface: method, auth, contract, and errors. . . . 35
5.5 Hyperparameters a rerun must hold fixed. . . . 36
5.6 Test inventory behind the delivered code (counts collected 2026-10-23). . . . 36

6.1 Retrieval: recall and MRR by arm. ACCA panels share the 30-question harness; the two endpoint rows are the live deployed path (plain RPC plus reranked); DDIA is the second corpus. . . . 43
6.2 Generation: delivered S:off arm, S-layer sweep aggregate, groundedness judge, and coding pair. Judge and citation metrics are distinct by construction. . . . 44
6.3 Knowledge tracing: AUC and calibration across three populations. Parity on ranking; calibration nearly halves on live attempts. . . . 44
6.4 Loop verdict: post-fix replay across four archetypes. Sufficiency 0.66 to 0.77 is the cross-user range; per-archetype decidedAt values sit in the versioned JSON beside the replay evidence. . . . 45

A.1 Adherence emission: actual over planned minutes by user and week. Week 5 is not observed (n.o.) for any user. . . . 51
A.2 Loop verdict instants: per-user decidedAt values. . . . 52
A.3 DDIA second-corpus gold set, questions 1 to 15. Snippets are verbatim from the evidence file. . . . 53
A.3 DDIA second-corpus gold set, questions 16 to 30. . . . 54

Definitions and Abbreviations

Term	Meaning in this report
AC1–AC4	Acceptance Criteria 1–4 for the live deployment check (Sec 6.2).
S-layer	Generation sweep aggregate (n=180). S:1–S:6 are the six distinct groundedness question keys; S:off is the delivered reasoning-off arm (Sec 6.1).
ECE	Expected Calibration Error for mastery tracing (Sec 6.1).
ACCA	Frozen ACCA study text: 754 chunks with 768-dimensional sidecar vectors; primary offline corpus (Sec 5.2, 6.1).
DDIA	Second corpus: 1094 chunks; confirms the retrieval direction (Sec 5.2, 6.1).
OULAD	Open University dataset; calibrates the Phase I generator only, outside the Phase II comparison (Sec 5.2).
Posture	Planted weekly behavior pattern per synthetic archetype: steady, crammer, struggling, returning (Sec 4.3, 5.2).
Sufficiency	Judge evidence-sufficiency score for the loop verdict; converted replans read 0.66–0.77 (Sec 4.1, 6.3).
Verdict pair	Exact verdict payload mirrored from `suggest_replan` onto `RoadmapReplanned` (Sec 4.1, 6.3).
Dip-window	Week-3 trough replay with the browser clock pinned; trough holds on the merits (Sec 6.4).
Met-class	Agreement-met subset in the Jev train/test split; the shipped cutoff rests on two pairs (Sec 6.1).
Top-up	Fresh live coding pull after the pre-top-up gold run, which reads 0.0 derivable (Sec 6.1).
Gate fix	Loop gate moved from 0.9 to 0.7 (`LOOP_APPROVE_AT` 0.7); confidence 1.0 would also clear the old gate (Sec 4.2.5, 6.3).

Chapter 1

Introduction

This chapter describes the problem this phase addresses and the shape of the work that follows. Section 1.1 recalls the Phase I system in one paragraph. Section 1.2 states the Phase II problem. Section 1.3 lists the objectives of Phase II, each with a short explanation. Section 1.4 defines the scope of the work across both phases. Section 1.5 maps the chapters that follow.

1.1 Background

A self-directed learner is a person who works through online courses, examination preparation, or self-study material without an instructor setting the pace. Such a learner typically begins with a fixed plan, for example two hours a day until a fixed finish date. Within days the plan no longer matches reality, yet the fixed finish date stays displayed as though it were still valid. Phase I of this project built an open-loop adaptive planner for this learner.

The system estimated study pace from session history with a Bayesian model with hierarchical priors and a context-aware shrinkage calibrator. The prior structure follows the adaptive-teaching model of Chen et al. [1]. It flagged candidate shifts in pace for learner confirmation with Cumulative Sum (CUSUM) change-point detection. It projected the completion date with an uncertainty band using Gaussian-process regression. It regenerated the schedule from realized capacity when the projection diverged from the deadline. Each component was compared against baselines on a synthetic dataset with known ground truth.

The comparison used held-out evaluation with bootstrap confidence intervals and multiple-comparison correction. The selected models were realized in a working local-first web application. The enriched-shrinkage calibrator beat the moving-average baseline on prospective error, including the short-history band (see the Phase-I evaluation). Change detection stayed a documented null when no candidate dominated the frontier. What the open-loop system never checked was whether the logged hours produced learning.

1.2 Problem Statement

The pace signal in Phase I comes from self-reported session logs. It records study time and says nothing about understanding. A learner can log the planned hours yet fail the assessment, while the

plan still reports pace as healthy. Existing tools cover the time side of the gap, not the understanding side.

Learning-analytics measures of self-regulated learning lean on time-management traces [2], so time on task is measured while understanding is not. The prediction-then-optimize study predicts grades and then optimizes preparatory leave [3], so it schedules from predicted grades, not graded evidence from the learner's own answers. The missing piece in the Phase I system is assessment-based verification of graded evidence. Grades update mastery, and the mastery verdict drives the decision to stay the course or to replan.

Phase II closes this loop. It generates grounded assessments from the learner's own materials. It grades them with an authoritative arm per question family. It tracks mastery from the grade history. It sends a verdict on the plan to schedule regeneration.

1.3 Objectives

Phase II of the project pursues the following objectives.

1. Generate grounded assessments in three families. Each generation call produces one question in a single family: objective, written, or coding. Every question is grounded in retrieved chunks of the learner's material with recorded citations. A mixed run alternates the family per call. Grounding and recorded citations keep every question traceable to the material it tests.

2. Grade each family with its authoritative arm. Objective questions grade deterministically against authored answers. A language model applies a rubric to written answers, and coding answers execute in a sandboxed runner with hidden tests. The server grade is authoritative. Any client-side check stays advisory and never reaches the record.

3. Estimate mastery with Bayesian knowledge tracing and recommend difficulty from it. A Bayesian model updates a per-skill mastery probability from the graded response history. The practice system uses the estimate to recommend the difficulty of upcoming problems instead of serving a fixed sequence.

4. Guide with a Socratic ladder and a gated reveal. The final step stays locked until the learner confirms, so guidance never gives the answer before the learner works.

5. Return advisory feedback copy on updated results. A server endpoint drafts learner-facing feedback grounded in the attempt and its citations for entries whose grades changed. It uses a static fallback when the model is unreachable. The copy never alters the grade.

6. Run mixed practice through the library and practice hub. Assessments and practice runs live under the material library with cache-first loading, and a practice run resolves its questions lazily from the loaded assessments. This keeps starting practice responsive on slow networks.

7. Replan from the loop verdict and validate the loop on synthetic longitudinal histories. A loop endpoint reads mastery, the schedule posture, and grade evidence. It returns a hold or

replan verdict under a fixed confidence gate of 0.7 (see Sections 4.2.5 and 6.3). A verdict for replan passes to schedule regeneration with its verdict pair attached. Four synthetic learner archetypes over five weeks each supply the longitudinal histories.

1.4 Scope of the Project

The work is delivered across two phases. **Phase I**, reported earlier, built and evaluated the open-loop system: pace calibration, change detection, completion projection, and capacity-based schedule regeneration. Each part was compared offline against baselines, and the selected models were realized in the application. **Phase II**, reported here, closes the loop with assessment-based verification of graded evidence: grounded generation, authoritative grading, mastery tracing, Socratic guidance, and advisory feedback. Mixed practice runs and verdict-driven replanning were validated on synthetic longitudinal histories.

The target user in both phases is the individual self-directed learner. Multi-user, institutional, and instructor-mediated settings remain out of scope, as does deployment in a real classroom. The synthetic archetypes replace the longitudinal real-learner study planned in Phase I. A real single-learner study remains future work.

1.5 Organization of the Report

The remainder of the report is organized as follows. Chapter 2 surveys the research this phase uses. Sections 2.1 to 2.4 cover knowledge tracing, item generation and tagging, grading, and guidance with feedback. Section 2.5 notes the sourcing method, and Section 2.6 summarizes the approaches and the gaps this project addresses.

Chapter 3 specifies the system requirements in Sections 3.1 to 3.4. Chapter 4 describes the proposed methodology. Section 4.1 presents the closed loop, Section 4.2 the selected models, and Section 4.3 the comparison method. Section 4.4 records the design evolution, and Section 4.5 presents the system architecture.

Chapter 5 gives the implementation details in Sections 5.1 to 5.5. Chapter 6 presents the results and discussion in Sections 6.1 to 6.7. Chapter 7 concludes in Section 7.1, describes future work in Section 7.2, and states the limitations in Section 7.3.

Chapter 2

Literature Survey

This chapter surveys the techniques this phase uses. Section 2.1 covers knowledge tracing and mastery models. Section 2.2 reviews automatic item generation and knowledge-component tagging. Section 2.3 describes grading methods for written and coded answers. Section 2.4 surveys Socratic guidance and learner feedback.

Section 2.5 notes the method used to source the surveyed work. Section 2.6 summarizes the approaches and states the gaps this project addresses.

2.1 Knowledge Tracing and Mastery

Tracing mastery from a short response history is the basis of the closed loop. Corbett and Anderson [12] introduce knowledge tracing on an intelligent programming tutor. Each production rule carries four parameters, and the model predicts test performance while individualizing the parameters per rule per student. The updater this project ships is that Bayesian updater. DKT [25] replaces the per-skill parameters with a recurrent network that learns a latent knowledge state from response sequences at scale. The learned state needs dense logs, so a single self-directed learner with almost no graded history stays a cold start.

Chen et al. [1] present a hierarchical Bayesian model of adaptive teaching and validate it through two online behavioral experiments with 312 adults. Hierarchical priors capture learner-to-learner variation without overfitting sparse data. This supports estimation of a single learner's state from few observations. The model targets a teacher's choice of examples rather than mastery of a tested skill. It supplies the prior structure but not the updater this project needs.

The updater is Bayesian Knowledge Tracing. It tracks a per-skill mastery probability through four fitted parameters: prior knowledge $P(L_0)$, learning rate T, guess G, and slip S. After a correct response the posterior follows

$$P_{\text{correct}}(L_n) = \frac{P(L_{n-1})(1-S)}{P(L_{n-1})(1-S) + (1-P(L_{n-1}))G}, \qquad P(L_n) = P_{\text{obs}}(L_n) + (1-P_{\text{obs}}(L_n))T. \quad (2.1)$$

The subscript names the observation just seen. The first term revises the estimate from the observed response. The second term moves the learner toward mastery by the fitted rate.

Zanellati et al. [4] review hybrid knowledge-tracing models in a taxonomy of approaches that combine Bayesian posteriors with machine learning. They organize hybrid models into a taxonomy

that combines Bayesian posteriors with machine learning.

Against this, the project's own bake-off reports its Bayesian variant, called bkt-v1, at an area under the curve (AUC) of 0.888. The running-proportion baseline reads 0.889. The comparison is the Chapter 4 bake-off (see Sections 4.3 and 6.1). That is parity rather than a win.

The project kept the simpler, per-skill updater at parity to serve an inspectable estimate, not a small ranking gain (see Section 4.2.3). The surveyed hybrids address correctness prediction on response logs. They do not steer the difficulty of what the learner sees next, which is the step this project adds with its adaptive chip.

Mai et al. [5] couple deep temporal modeling with a probabilistic layer in a Transformer-Bayesian hybrid that recovers interpretable, calibrated knowledge estimates. The authors present this as keeping transparency without giving up accuracy on their evaluation. The limitation for this project is cold start. A single self-directed learner arrives with almost no graded history. Phase I found that a calibrated pace estimator can win on short session histories. Mastery tracing has a harder cold start because it needs graded responses, not session logs.

The project therefore keeps the Bayesian updater with the fixed prior set by the bake-off. The mastery estimate then feeds an advisory difficulty recommendation instead of a fixed sequence.

Calibrated probabilities behind such estimates rest on older machinery. Niculescu-Mizil and Caruana [26] show that classifier scores become reliable probabilities only after calibration, comparing Platt scaling and isotonic regression on boosted trees and SVMs. Psychometrics reaches the same point from the other side. Embretson and Reise [22] build item response theory. There the probability of a correct response is a function of learner ability and item parameters. Both traditions say the same thing this project assumes: a rank order is not a probability until it is calibrated.

The live harness therefore reports ECE beside AUC, and the adaptive chip stays advisory while its calibration is unmeasured.

Once mastery is estimated, practice must be scheduled. Cepeda et al. [19] synthesize 839 distributed-practice assessments from 317 experiments and find that the interstudy interval that maximizes retention rises with the retention interval. Spacing works, but the schedule depends on the horizon. Settles and Meeder [20] make the schedule trainable. Their half-life regression on Duolingo logs cuts recall-prediction error by over 45% against baselines. It lifts daily engagement by 12% in an operational study.

Neither study steers what the learner sees next from a mastery estimate. The adaptive chip in this project starts from that estimate instead of a fixed sequence.

The Phase I estimators behind this loop rest on three older methods. Page [13] introduces the cumulative-sum change-point procedure the pace monitor uses. Rasmussen and Williams [14] give the Gaussian-process regression behind the completion projection with its uncertainty band. Vovk et al. [15] give the conformal prediction behind the split-conformal interval correction, which stays a validated but unshipped refinement. None of the three checks learning. They bound the plan the loop then verifies.

2.2 Automatic Item Generation and KC Tagging

A knowledge component (KC) is the single skill a question tests. The loop needs fresh questions on the learner's own material, each traceable to the passage it tests. The large-language-model prompting study across STEM subjects [6] systematically investigates conceptual understanding and the

quality of generated question-answer pairs across STEM subjects. Against this, the project's own generation harness reports a groundedness of 0.8667 with a near-duplicate rate of 0.0044 (26 of 30 judged, see Section 6.1). The paper evaluates generation per subject but does not record which chunk each question came from. A wrong question therefore cannot be traced back to its source.

The lecture-to-quiz study [7] uses a local model to generate multiple choice questions. It checks them with deterministic quality control. Across 15 runs it accepts 120 of 122 candidates while the warning layer flags 8 of 120 for residual risks such as duplicated distractors. The control step rejects malformed items before they reach the learner.

The project's retrieval arm behind generation uses the `hybrid_rerank` configuration on the ACCA 754-chunk split (n=30, see Section 6.1). It reaches recall at 3 of 0.90 with a mean reciprocal rank of 0.797, against a hybrid baseline of 0.80 and 0.720. Deterministic checks reject malformed items with missing options or citations before they reach the learner. They cannot catch a well-formed question grounded in the wrong passage. Each generation call in this project therefore produces one question in a single family and records its chunk citations.

Tagging items to knowledge components faces the same split. The automated tagging study [8] generates knowledge-component tags from multiple-choice questions and attaches each item to the component it tests. Their best strategy matches human tags on 56% of Chemistry and 35% of e-learning items. The tag makes per-skill histories possible, but the study does not use it to steer practice.

The project's mastery model needs exactly that signal, and its bake-off holds at the parity noted in Section 2.1. This project therefore carries the per-problem skill tags from generation into the run. Practice runs resolve their questions lazily from the loaded assessments instead of copying a fixed list.

2.3 Large Language Model (LLM) and Sandbox Grading

A grade that steers the plan must come from an authority the learner cannot sway. Retrieval practice is the reason the grade carries weight at all. Roediger and Karpicke [29] show that taking memory tests improves long-term retention beyond restudying. Dunlosky et al. [30] rate practice testing and distributed practice as the two highest-utility learning techniques. A graded attempt is therefore evidence twice over: it measures the state and it changes it. The loop records the measurement and lets the plan respond to it.

The EDUCON study on short-answer grading [9] applies large language models to grade short answers in software engineering courses automatically. A rubric is intended to make the judgment repeatable across answers.

A model judge needs its own validity evidence. Zheng et al. [21] show that a strong LLM judge exceeds 80% agreement with human preferences on MT-Bench and Chatbot Arena and matches human-human agreement. It documents position, verbosity, and self-enhancement biases.

Kasneci et al. [31] survey the same bargain for education: fluent model output with hallucination, bias, and privacy risks that need layered validation.

This project therefore never lets the judge stand alone. The groundedness judge is disclosed as self-judged. The written arm keeps a calibrated cutoff with a human-auditable margin, and hidden tests decide code.

The project's deployment check is separate: it graded written runs 4 of 4 with real rubric grades

(see Section 6.2). A rubric grade alone cannot carry code. An answer there is right only if it runs. The project therefore keeps the model-judged arm for written answers and a separate execution arm for code.

Deterministic grading covers the objective family. The answer is authored at generation time and the grader compares it exactly, with no model call. The adjacent prediction-then-optimize study [3] predicts grades before optimizing preparatory leave on its data, which shows that grade-adjacent signals can inform scheduling. Prediction informs scheduling, but it does not produce the grade the loop records. The project's live matrix takes objective runs 2 of 2 and the review queue passes 4 of 4 (deployment check, see Section 6.2). The server grade stays authoritative, while any client-side check stays advisory and never reaches the record.

Code answers execute in a sandboxed runner with hidden tests. The runner is the self-hosted Piston service. The project grades coding runs 3 out of 3 live (deployment check, see Section 6.2). The hidden pass rate decides the grade, and the reference solution never reaches the browser. The client runs visible tests only as advice.

The gap this design answers is single-signal grading treated as final. One arm cannot judge three families, so each family gets its own authoritative arm and hidden material stays server-side.

2.4 Socratic Guidance and Feedback

The closest prior systems are intelligent tutors, and the gap to them is specific. Bloom [27] poses the 2-sigma problem: one-to-one tutoring lifts performance by two standard deviations over classroom instruction, which no school can staff. Cognitive Tutors [18] answer with model tracing plus knowledge tracing plus mastery learning. The tutor follows the learner step by step, updates per-skill mastery, and advances each skill at mastery. VanLehn [16] reviews the experiments and finds step-based tutors at d=0.76 against d=0.79 for human tutoring: nearly as effective.

The loop in this project is that architecture for one self-directed learner without a classroom. It uses generated items instead of authored curricula, a server grade instead of step tracing, and a replan instead of a teacher. It does not reproduce step-level tutoring, and it claims no learning gain.

How much help a tutor gives is itself contested. Koedinger and Aleven [17] name the assistance dilemma from Cognitive Tutor experiments. Giving information helps immediate performance, while withholding it can improve robust learning. The balance stays an open instructional problem. The gated ladder in this project takes the withholding side by construction. The final step stays locked until the learner confirms, so guidance never gives the answer before the learner works.

Guidance that gives the answer destroys the evidence the loop needs. Alhazbi et al. [2] review 25 empirical studies on measuring self-regulated learning from trace data and find that most validated indicators relate to time-management skills. This grounds schedule adherence and regularity as behavioral metrics. The review measures behavior. It does not guide the learner through a hard problem, which is where the Socratic ladder in this project starts.

Chi and Wylie [24] order engagement from passive to constructive to interactive and predict deeper learning at the higher modes. The ladder aims at the constructive mode by forcing the learner to produce each step.

Uzun et al. [10] find that high self-regulated-learning competence goes with more active feedback engagement, but that engagement does not significantly move course performance. The lesson for this project is that feedback must be read to matter, and reading alone does not lift grades.

The project's guide runs 2 out of 2 live with real streamed steps (deployment check, see Section 6.2). In this design, ungated feedback risks drifting into answer-giving, so each hint step stays gated.

The project therefore answers through a ladder of Nudge, Hint, Targeted explanation, and Reveal. The last step stays locked until the learner confirms, so guidance never gives the answer before the learner works.

Help-seeking itself can be gamed. The think-aloud study of gaming the system [11] analyzes self-regulated learning protocols and separates genuine attempts from moves that extract answers from the tutor. The project's replay measures a different outcome. After the fix, the synthetic histories convert 4 of 4 verdict-triggered replans, against 4 of 4 holds before the fix. The replay ran at gate 0.7 (see Section 6.3). The material-scoped span is 32 days.

The feedback copy in this project stays advisory for the same reason. Hattie and Timperley [23] synthesize prior meta-analyses into a model of effective feedback. It answers where the learner goes, how the learner goes, and where to next. Shute [28] adds that formative feedback should stay specific, timely, and non-evaluative. A server endpoint drafts learner-facing text for entries whose grades changed, with a static fallback, and the copy never alters the grade.

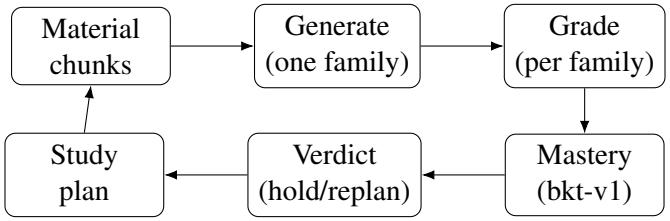


Figure 2.1: The assessment-verified closed loop this chapter motivates: generated items are graded per family, grades update mastery, the verdict steers the plan, and the plan points back at the material. Each stage is motivated in Sections 2.1 to 2.4, and the sandbox and feedback-copy stages are implementation-only.

2.5 Methodology for Sourcing the Surveyed Work

The survey above combines two passes. The first pass is advisor-guided curation with citation chaining from the Phase I Bayesian learner-modeling sources and the implementation anchors of Phase II. These are single-family generation, per-family grading arms, the Socratic ladder, and advisory feedback. Each candidate was checked against four clusters matching Sections 2.1 to 2.4. Coverage across the four clusters mattered more than raw count. One to three representative works were kept per sub-theme, and redundant near-duplicates were dropped.

The second pass is a targeted verification sweep run on 2026-10-23. It covers publisher records, the ACM Digital Library, the ACL Anthology, and Scholar-indexed records. About 20 targeted queries corroborated the title, venue, year, and where used the reported metric of every cited work. Queries named the paper title with the venue or DOI, for example the ICCIT preparatory-leave paper, the L@S knowledge-component paper, and the EDUCON short-answer paper. Inclusion required a corroborated venue and year. A candidate whose metadata could not be corroborated was dropped.

No hundred-scale database screening ran, and none is claimed. Scopus and IEEE full-text screening with recorded inclusion counts remain author follow-up. The 29 retained references split into 11 Phase I and Phase II anchors and 18 added records. They cover estimation foundations, tutoring systems, spacing, judge validity, psychometrics, and feedback.

No new hundred-scale screening ran for this chapter, and none is claimed. Paper-side metrics appear where the source gives them verbatim, as in Sections 2.1 and 2.4. Where the Phase-II sources state scope without a reusable number, the chapter states the scope. Where the project has a comparable number, it appears beside the paper's and is labeled as the project's. This keeps every project metric traceable to the Phase I report or to live code. Paper-side numbers are cited where the source states one.

2.6 Summary of Approaches

Table 2.1 summarizes the families with their strengths and the gaps that motivate this work. Each technique is strong in isolation. None forms the combination this project builds. That combination is single-learner local-first planning, verified by generated grounded assessments, graded per family on the server, with verdict-driven replanning.

Table 2.1: Summary of approach families, with strengths and limitations for this project.

Approach family	**Strengths or design basis**	**Gap this design addresses**
Knowledge tracing [12, 4, 5, 25]	Per-skill mastery from short histories, interpretable, calibrated	Predicts correctness, does not steer difficulty
Calibration and psychometrics [26, 22]	Reliable probabilities, ability-plus-item models	Rank orders used as probabilities without calibration
Estimation heritage [13, 14, 15]	Change detection, projection bands, interval correction	Bounds the plan without checking learning
Item generation and KC tagging [6, 7, 8]	Fluent items at scale, deterministic checks, component tags	Untraceable to source chunks, tags unused for steering
Rubric grading [9] and implementation-only sandbox execution	Rubric repeatability, deterministic match, hidden-test authority	Single-signal grades treated as final, client checks leak answers
Judge validity [21, 31]	80% human agreement with documented biases, layered validation	Unaudited judges treated as ground truth
Tutoring systems [27, 18, 16, 17]	Near-human tutoring effects, mastery learning, assistance design	Classroom systems, no local-first single-learner loop
Spacing and retrieval [19, 20, 29, 30]	Durable retention, trainable schedules, testing as learning	Schedules and tests unused for plan steering
Guidance and feedback [24, 2, 10, 11, 23, 28]	Constructive engagement, validated indicators, feedback the learner reads	Answer-giving tutors, grade-altering feedback

Sandbox execution is implementation-only. The table claims no surveyed source for it.

The gaps follow from the table. First, generation is fluent but untraceable, and tags exist without steering, so questions cannot yet drive per-skill practice. Second, grades come from one signal or from the client, so they cannot yet carry plan decisions. Third, guidance either measures without helping or helps by giving away the answer, so attempts stop being evidence.

The closest prior work is the Cognitive Tutor line [18, 17]: model tracing plus knowledge tracing plus mastery learning, reviewed at near-human tutoring effects [16]. This project is that architecture narrowed to one self-directed learner without a classroom. It uses LLM-generated grounded items instead of authored curricula, per-family server grades instead of step tracing, and a verdict-driven replan instead of a teacher. It does not reproduce step-level tutoring, and it claims no learning gain.

This project answers all three gaps with the loop in Figure 2.1. Generation is grounded and cited, and each family carries its own authoritative grade. Difficulty follows mastery, the ladder stays gated, and feedback stays advisory. The verdict feeds schedule regeneration in Chapter 4.

Chapter 3

System Requirements Specification

This chapter specifies the requirements of the full system. Sections 3.1 and 3.2 state the hardware and software environment. Section 3.3 lists FR-1 to FR-15, and Section 3.4 lists NFR-1 to NFR-9. FR-9 to FR-15 and NFR-7 to NFR-9 cover the Phase II additions.

The chapter assumes a single self-directed learner, a workstation with a graphics processor, and access to the hosted model APIs. It also assumes that synthetic longitudinal histories replace a classroom study, as stated in Section 1.4.

3.1 Hardware Requirements

The system runs on a single workstation. A single learner produces a small volume of data with tens to hundreds of sessions. Statistical models train and run on the CPU. Embedding and rerank calls run on the graphics processor, so the workstation includes a discrete graphics processor. Table 3.1 lists the minimum hardware environment. A note under the table records the project machine.

Table 3.1: Hardware environment.

Component	**Minimum requirement**
Processor	x86-64 CPU with a discrete graphics processor
Memory	8 GB RAM or higher
Storage	At least 60 GB free for the container images, before build cache and volumes
Container runtime	Docker to run the intelligence service, the worker, the GPU inference sidecar, and the sandbox
Client device	Any device with a current browser that supports IndexedDB and ES2022

The project machine uses an AMD Radeon RX 9070 XT and has 19 GB of RAM. Its container images measure about 53 GB: 49.5 GB for the GPU sidecar, 1.42 GB for Piston, and about 1.7 GB

for the product images. Build cache and volumes bring the total to about 74 GB, and browser data stays within the browser's per-origin quota.

3.2 Software Requirements

The system keeps the Phase I tiers and adds the GPU inference sidecar plus the code-execution sandbox. The browser client stays local-first. The Python intelligence service and its worker now host generation, grading, mastery, guidance, and feedback. Supabase still provides authentication, the event database, and object storage. The GPU inference sidecar serves embeddings and rerank scores, and the sandbox runs learner code. Table 3.2 lists the software environment by tier.

Table 3.2: Software environment by tier.

Tier	Technologies
Client (single-page app)	React 19, Vite 5, TypeScript, Dexie over IndexedDB for local-first storage, date-fns
Intelligence service and worker	Python 3.12, FastAPI, Uvicorn, NumPy, PyJWT, generation, grading, mastery, guidance, and feedback endpoints, packaged with Docker
Model providers	OpenRouter generation and rubric-grading provider (`deepseek/deepseek-v4-flash-0731`), plus the Jev-1.13 advisory judgment client
Backend services	Supabase: authentication, a PostgreSQL event database including `jev_review_flags` for the written-arm review queue, and object storage
GPU inference sidecar	Qwen3-Embedding-0.6B plus Qwen3-Reranker-0.6B behind `/embed` and `/rerank`, separate compose project
Code-execution sandbox	Piston (`ghcr.io/engineer-man/piston`) on port 2000 for hidden-test grading
Offline research pipeline	Python (uv workspace), NumPy, with pytest and ruff for testing and linting
Tooling and deployment	pnpm workspace, Vitest and Playwright for testing, Vercel for hosting

3.3 Functional Requirements

The functional requirements state what the system does. Items FR-1 to FR-8 carry the Phase I planner. Items FR-9 to FR-15 add the assessment-verified loop.

FR-1 The system shall let a learner define a study plan from a set of materials, a deadline, and a weekly study capacity.

FR-2 The system shall log study sessions, both from an in-application timer and from manual entry of a past session.

FR-3 The system shall estimate the learner's pace from the logged sessions using hierarchical Bayesian calibration.

FR-4 The system shall detect a shift in the learner's pace and surface a recalibration prompt for the learner to act on.

FR-5 The system shall project the completion date with an uncertainty band rather than a single date.

FR-6 The system shall regenerate the schedule from the learner's realized capacity when the learner chooses to replan.

FR-7 The system shall present the learner's progress, including the burn-up curve, streak, and schedule adherence.

FR-8 The system shall persist all data on the device and synchronize it to the cloud when a connection is available.

FR-9 The system shall generate one grounded question per call in a single family, with each question carrying its recorded chunk citations.

FR-10 The system shall grade each family with its authoritative arm. Deterministic grading applies to objective items. Rubric grading with the OpenRouter rubric model applies to written items. Jev-1.13 is an advisory shadow and the loop-verdict judge. Hidden-test pass rate applies to coding items.

FR-11 The system shall estimate mastery per skill with the `bkt-v1` model and provide an advisory difficulty recommendation from that estimate. The recommendation is not calibrated (see Section 6.1).

FR-12 The system shall guide the learner through a Nudge, Hint, Targeted explanation, and Reveal ladder and keep the full answer gated behind an explicit reveal.

FR-13 The system shall supply advisory feedback copy on updated results only, and that copy shall never alter a grade.

FR-14 The system shall run mixed practice sessions through the assessment library and the practice hub, with pause as leave-and-return.

FR-15 The system shall replan from the loop verdict once assessment evidence shows the plan is off track. Logged hours alone do not trigger a replan (see Section 4.1 for the hold rule and Section 6.3 for the replay evidence).

3.4 Non-Functional Requirements

The non-functional requirements state the qualities the system must exhibit. Items NFR-1 to NFR-6 carry the Phase I qualities, with NFR-2 restated from the Phase I CPU-only form to cover the single-GPU sidecar. Items NFR-7 to NFR-9 add the measurable Phase II qualities.

NFR-1 **Offline-first operation.** The core logging loop shall function without network connectivity, and analytical results shall be cached locally for reuse.

NFR-2 **Single-GPU scale.** The system shall serve the individual-learner workload on one workstation graphics processor, with no cluster and no multi-GPU setup.

NFR-3 **Per-learner isolation.** Each learner's data shall be isolated, using a per-user local database and row-level security in the cloud database.

NFR-4 Authenticated services. Requests to the intelligence service shall carry a signed token that is verified against the identity provider before any model runs.

NFR-5 Responsiveness. Calibration results shall return within 8 s per attempt, with at most two retries, so the worst case is three attempts. Projection runs locally without a network call. The client keeps the last cached result with a stale flag when the service is unreachable.

NFR-6 Behavioral alignment. Where an algorithm exists in both the client and the service, the two implementations shall stay behaviorally aligned on versioned golden fixtures. Calibration and roadmap engines replay the same fixtures in TypeScript and Python with numeric equality to a tight tolerance.

NFR-7 Grounded generation. The judged groundedness of generated questions shall be at least 0.80 on a 30-question sample (the Section 6.1 arm reads 0.8667).

NFR-8 Grade authoritativeness. Hidden tests, reference solutions, and authored accepted values shall never reach the browser, and the server grade shall be final while client results stay advisory.

NFR-9 Retrieval quality floor. Retrieval shall hold recall at 1 of 0.60 and recall at 3 of 0.85. It shall hold a mean reciprocal rank of 0.75 on the frozen ACCA 754-chunk corpus. The floor sits below the `hybrid_rerank` arm (0.67, 0.90, 0.797 on n=30, see Section 6.1).

NFR-7 and NFR-9 are prospective floors with headroom below the measured arms, not the measured values themselves.

3.5 Product Perspective

The product is a local-first web application. A TypeScript client with a per-user browser store talks to a Python intelligence service with a portless worker. Supabase is the auth, database, and storage hub. Phase I delivered the planner substrate of session runtime, progress views, and roadmap lifecycle, and that substrate is unchanged. Phase II adds assessment generation, per-family grading, mastery tracing, Socratic guidance, advisory feedback, mixed practice, and the loop verdict on top of it.

The only actor is the single self-directed learner. Multi-user, institutional, and instructor-mediated settings are out of scope. External interfaces are the browser, the Supabase project, the OpenRouter API for generation, grading, and judging, and the two sidecars for embeddings and code execution.

3.6 Assumptions, Dependencies and Constraints

The work assumes one learner with their own materials in text, PDF, or web-article form. The learner runs a modern browser with local storage on a steady network for service calls. It depends on a Supabase project for auth, database, and storage. It also needs an OpenRouter key for the generation, grading, and judge models, the Qwen3 sidecar models, and the Piston image for code execution.

Generation and grading call a third party, so learner material leaves the device. The report states no retention contract. The build constrains deployment to a single workstation graphics processor with no cluster. The loop verdict constrains replans to evidence the grades support.. The

build constrains deployment to a single workstation graphics processor with no cluster, and the loop verdict constrains replans to evidence the grades support.

3.7 Operating Environment

The stack runs containerized. Web, intelligence, ingestion worker, and Piston form the product compose project. The Qwen3 embedding and rerank sidecar has its own compose project on port 8200. Local development runs the Python 3.12 service on port 8000 and the React client on port 5173. The sandbox profile on port 2000 runs under Docker Desktop on WSL2.

The client needs only a modern browser with IndexedDB. All heavy compute stays server-side or in the sidecars.

Chapter 4

Proposed Methodology

This chapter describes how the closed loop works and why it takes this shape. Section 4.1 presents the closed learning loop, and Section 4.2 states the candidate models and the selected approach. Section 4.3 describes the comparison methodology. Sections 4.4 and 4.5 trace the design evolution and present the system architecture.

4.1 The Closed Learning Loop

The loop runs over five stages. The learner plans through roadmap-engine bookings and studies through logged sessions. Practice and assessment produce server-graded evidence. Mastery and the loop verdict recalibrate the plan state. The verdict then drives the replan. Figure 2.1 shows the same loop at concept level.

The verdict is advisory and passes a code gate. `POST /v1/loop/decision` asks the hosted judge for one choice over the week's signals, the evidence summary, and the schedule posture. The code router holds evidence below 0.5 and confidence below 0.7 on `hold`. Only `suggest_replan` carries the verdict pair forward to the replan screen, and the committed `RoadmapReplanned` mirrors it. `Keep` writes no event.

Phase I ran this loop open. It recalibrated pace from logged sessions and raised a prompt, and the learner decided whether to replan. Nothing checked that study produced understanding. Phase II verifies first: grades steer mastery through Equation 2.1, mastery and posture steer the verdict, and the verdict steers the plan.

4.2 Candidate Models and Selected Approach

The project fixed each stage by comparing candidates against evidence instead of by default. This section states the candidates and the selected approach per stage. Section 4.3 describes how the comparison ran. Table 4.1 summarizes the selections.

4.2.1 Item Generation

The generator had to produce questions the project can trace. The project rejected multi-family single-call generation without recorded provenance. The selected shape is one family and one question per call, grounded in retrieved chunks with recorded citations. The server gate enforces it: `formats` must be one of `objective`, `written`, or `coding`, and `questionCount` must be `1`. A mixed run alternates families across calls instead of mixing them in one call.

Input is one material, one family, and one scoped chapter. Output is one question with recorded chunk citations, or a terminal warning such as `code_not_derivable` that invents nothing. Format failures drop as `malformed_output` after at most one repair. Missing citations drop as `citation_missing`. An unverified quote keeps its citation with a `citation_unverified` flag.

4.2.2 Grading

One signal cannot judge three families, so each family carries its own authoritative arm. Objective answers are authored at generation time and compared exactly. Written answers go to a rubric-judged model arm. The provider scores each authored criterion, and the module composes the scores with the authored weights at four-decimal precision. The same judgments always give the same score.

An unusable rubric or an empty answer fails closed without retry. A provider payload that does not map onto the rubric returns for redelivery. Code answers execute in the Piston sandbox against hidden tests, and the reference solution never reaches the browser. Every client-side check stays advisory and never enters the grade record.

4.2.3 Mastery and Difficulty

The mastery candidates were the deep and hybrid variants surveyed in Section 2.1. The bake-off held the Bayesian variant at parity with the baseline, so the project kept it and named it bkt-v1. Among the deep and hybrid candidates, the per-skill updater was the simplest at parity. Its estimate is inspectable. The estimate updates per skill through Equation 2.1.

The shipped parameters are $p_{\text{init}} = 0.15$, $p_{\text{learn}} = 0.1$, $p_{\text{slip}} = 0.05$, $p_{\text{guess}} = 0.2$. Input is the graded response sequence per skill. Output is the mastery probability after each observation with the predicted correctness before it. The TypeScript engine and the Python mirror replay the same golden fixtures with numeric equality to 10^{-6}. The adaptive chip then maps the estimate to an advisory, uncalibrated difficulty recommendation (see Section 6.1).

4.2.4 Guidance and Feedback

Guidance that gives the answer destroys the evidence the loop reads. The project rejected an open tutor that reveals on demand. The selected shape is the ladder of Nudge, Hint, Targeted explanation, and Reveal, with the last step locked until the learner confirms. The ladder is a state machine: each step streams against retrieved chunks, a request advances one step, and only an explicit confirmation opens Reveal. No transition writes a grade or reaches the grade record.

Feedback copy is also advisory. A server endpoint drafts learner-facing text for updated entries through the `feedback-v2` template with a static fallback, and the copy never alters the grade.

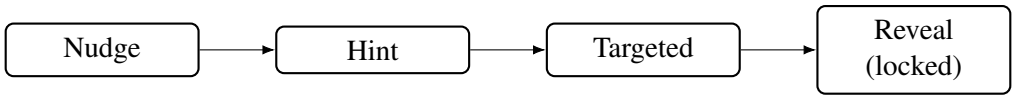


Figure 4.1: Guide ladder state machine: each step streams against retrieved chunks and only an explicit confirmation opens Reveal.

4.2.5 Loop Verdict

The project weighed a raw model choice against a gated router over it. The project kept the gated pair: the Jev judge (`jev-1.13` through a hosted API) proposes, and the code router disposes. Input is the week's signals with the evidence summary and the schedule posture. Evidence below 0.5 and confidence below 0.7 land on `hold`. The gate constant `LOOP_APPROVE_AT` is `0.7`. Only a confident call with sufficient evidence passes it.

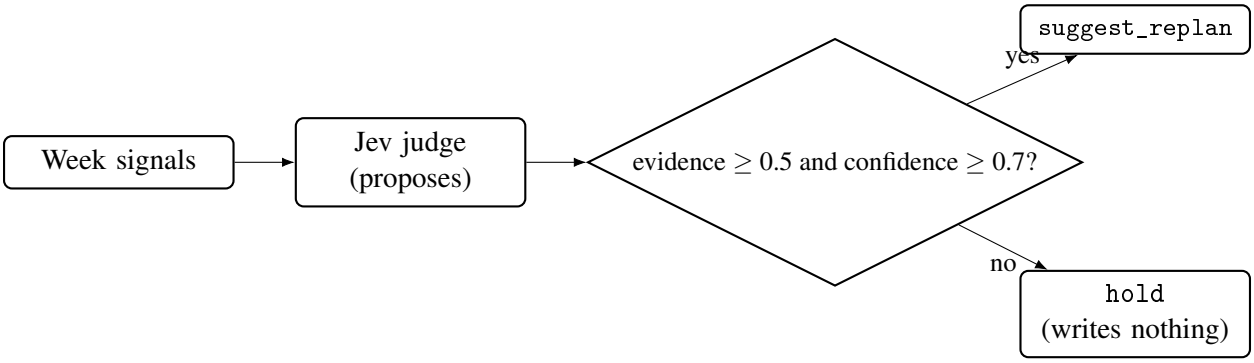


Figure 4.2: Loop verdict router: the judge proposes over the week signals and the code gate disposes.

Four limits stay on record. The coding arm refuses material it cannot ground a question in, and reports `code_not_derivable` instead of inventing a question. Difficulty calibration is unmeasured. Next-week adherence is unobserved. The split-conformal interval correction stays a validated refinement that is not shipped. The Phase I estimators carry over unchanged: the single-arm CUSUM still prompts recalibration.

4.3 Comparison Methodology

The comparison rests on two legs. The offline leg scores components against fixed golds. The longitudinal leg scores the whole loop on synthetic histories with known posture. The offline leg informed per-stage selection, and the longitudinal replay validated the assembled loop after the gate fix. The live browser matrix only checks deployment.

The offline leg is the generation-quality harness. It holds golds in `eval_golds.json` with three fold hashes at $K = 5$, and its gate stays fail-closed by default. It reports groundedness at 0.8667 with a near-duplicate rate of 0.0044. The harness's plain retrieval arm reports recall at 3 of 0.733, and the selected `hybrid_rerank` arm reads 0.90 on the 754 split.

On the separate mastery bake-off, bkt-v1 reads 0.888 against 0.889. The live harness reads 0.7397 against 0.7368. Coding derivability on the golds reads 0.0 on the pre-top-up run.

Table 4.1: Candidate approaches per loop stage and the approach selected for the delivered system.

Stage	**Candidates compared**	**Selected (delivered)**
Item generation	Multi-family single call, ungrounded prompting, single-family grounded	Single family and single question per call, chunk citations recorded
Grading	Advisory client-side check, single-signal grade	Server-authoritative arm per family. Hidden tests never reach the browser
Mastery and difficulty	Deep variants, hybrid models, fixed sequence	bkt-v1 with the adaptive difficulty chip
Guidance and feedback	Open tutor, grade-altering feedback	Gated ladder with locked reveal. Advisory-only `feedback-v2` copy with v1 fallback
Loop verdict	Raw model choice	Jev `jev-1.13` with the fail-closed router (`LOOP_APPROVE_AT` 0.7)

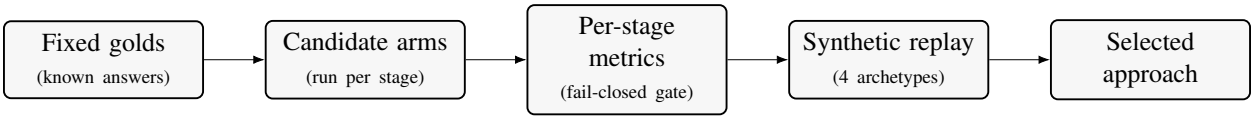


Figure 4.3: The two-leg comparison. Fixed golds score each candidate arm behind a fail-closed gate; the synthetic replay then validates the assembled loop after the gate fix.

The longitudinal leg replays four synthetic archetypes: steady, crammer, struggling, and returning. The replay tests the planted posture of the four synthetic histories rather than learner behavior. Each run ends in a loop verdict. A `suggest_replan` verdict converts when its exact verdict pair joins a `RoadmapReplanned`. Next-week adherence is reported separately.

The post-fix replay converts 4 of 4 `suggest_replan` verdicts into verdict-linked replans over a material-scoped span of 32 days. Verdict confidence reads 1.0, and the judge's evidence-sufficiency score ranges 0.66 to 0.77. Each of the four converted replans selected `extend-deadline`.

The live matrix confirms deployment. It never promotes a candidate. Objective runs take 2 of 2, and written runs grade 4 of 4 with real rubric grades. Coding runs grade 3 of 3.

Table 4.2: Evaluation metrics by loop stage.

Stage	**Metrics**
Item generation	Groundedness in the cited chunks, near-duplicate rate
Grading	Live pass counts per family (objective 2 of 2, written 4 of 4, coding 3 of 3)
Mastery and difficulty	Area under the curve against the baseline
Guidance and feedback	Live pass counts (guide 2 of 2, feedback copy 3 of 3). Copy never alters the grade
Loop verdict	Verdict-to-replan conversion (4 of 4 post-fix), next-week adherence not observed

4.4 System Design and Evolution

The system reached this shape in three moves. The starting point is the delivered Phase I design: capacity-based bookings with an open loop. Closed-loop feedback from verified mastery stayed a dashed extension in that design. The base scheduling study [3] still supplies the predict-then-optimize contrast that design answered.

The second move inserted assessment without touching the booking model. Generation, grading, mastery, guidance, and feedback landed in the intelligence service beside the Phase I calibration routes. The booking model and session logging did not change, but the local store gained Phase II attempt and mastery tables. The third move closed the loop through the verdict gate and the verdict pair mirrored onto `RoadmapReplanned`. The synthetic replay shows that `suggest_replan` converted to a verdict-linked replan in all four synthetic histories.

Section 1.2 states the problem, and Sections 3.3 and 3.4 state the requirements.

4.5 System Architecture

The deployed system keeps the Phase I split. Computation that fits a model to history runs in the Python service. Computation that derives a view from the local event log runs in the client. Supabase sits between them as the data hub with row-level isolation per learner. Figure 4.4 shows the tiers with the Phase II additions solid.

The browser client owns bookings, projection, mastery readers, and the verdict handoff. The intelligence service with its worker owns generation, grading, mastery, guidance, feedback, and the loop verdict. The Jev judge runs as a hosted-API client inside the service instead of a separate tier. The GPU inference sidecar serves embeddings and rerank scores from its own compose project, and the sandbox runs learner code on demand. Table 3.2 lists the full tier inventory.

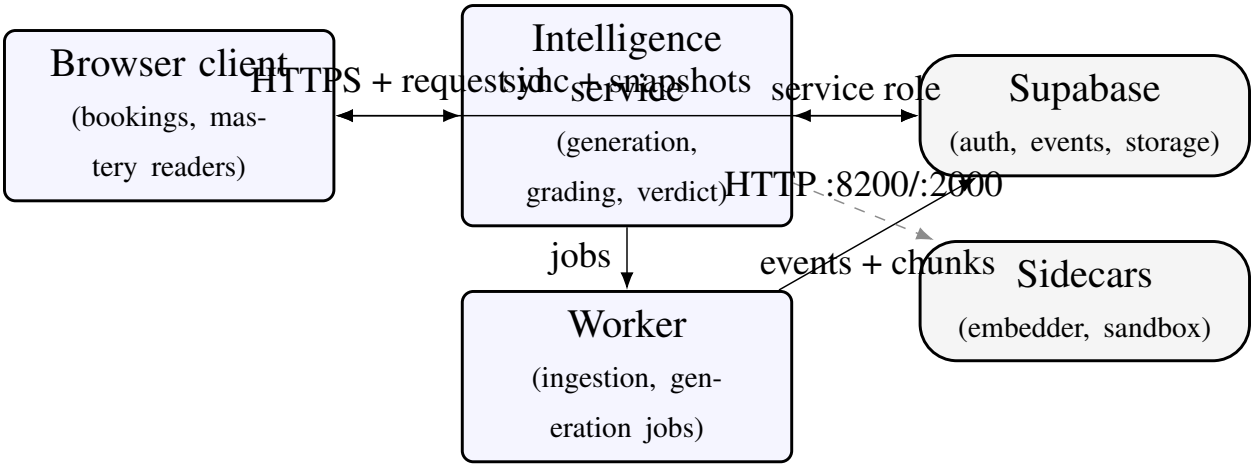


Figure 4.4: The deployment that realizes Figure 2.1. The client derives views from the local log, the service fits models to history with the judge folded inside, Supabase acts as the data hub, and the sidecars serve on demand.

Figure 2.1 states the concept the literature motivates, and this figure states where each stage runs and which wire joins them. The closed loop now runs end to end.

The durable product history is the per-user event log. Table 4.3 lists the Phase II event types with their writers and readers. The browser keeps one Dexie database per signed-in user. Schema version 7 covers events, sync queue and checkpoints, draft and session rows, attempt and content caches, and mastery cache. Only the event and queue tables sync. Caches rebuild from durable grades.

Remote tables, storage paths, and job rows are user-scoped with ownership checks. Retrieval stays owner-scoped to the learner's own chunks.e user-scoped with ownership checks, and retrieval stays owner-scoped to the learner's own chunks.

Table 4.3: Phase II event types: writer, payload, and reader.

Event	Writer and payload	Reader
`SessionLogged`	Session runtime, material-scoped date and duration	Pace calibration, verdict posture
`PracticeRunStarted`, `PracticeRunFinished`	Practice hub run pointers, no question ids	Practice hub resume, never a grade
`QuestionAttempted` plus grade	Taker plus grading worker, per-question grade	Mastery updater, evidence summary
`RoadmapReplanned`	Verdict pair mirrored from `suggest_replan`	Schedule regeneration, plan identity

Chapter 5

Implementation Details

This chapter describes how the Phase II system is built. Sections 5.1 and 5.2 record the development environment and the datasets. Section 5.3 describes the model and service implementation. Section 5.4 records how the metrics are kept reproducible. Section 5.5 presents the Phase II application.

5.1 Development Environment

The project keeps the Phase I repository shape. A pnpm workspace holds the TypeScript client with its shared engines. A uv workspace holds the Python packages, the service, and the research code. The two coexist so that the research code imports the same Python engines the service runs.

Table 5.1 lists the environment. The first eleven rows carry the Phase I stack unchanged. The last seven rows add the Phase II generation, grading, feedback, judgment, sandbox, embedding, and editor concerns.

5.2 Datasets and Their Sources

A single learner's real log does not supply a known ground truth for latent mastery and pace in this design. The Chapter 6 comparison therefore runs on fixed artifacts with planted and known properties. Table 5.2 lists the datasets and the role each plays.

The frozen corpus is a single textbook (edition needs verification) at 754 chunks with 768-dimensional sidecar vectors. Three gold files score the offline leg against it: the objective probe set, the written gold set, and the coding gold set. The fixed folds and the attempt gate sit beside them in `eval_golds.json`.

The longitudinal leg replays four synthetic archetypes with known posture: steady, crammer, struggling, and returning. Their generator plants weekly pace, skip behavior, gap weeks, and replan back-pointers explicitly. The Open University dataset (version needs verification) calibrates the Phase I generator only and does not enter the Phase II comparison. Real single-learner logs stay reserved for future validation and are not used here.

Table 5.1: Development environment: tools and libraries. The first eleven rows carry the Phase I stack; the last seven add the Phase II loop.

Concern	Technology
Client language and build	TypeScript, Vite, pnpm workspace
Client UI	React 19
Local-first storage	Dexie over IndexedDB
Intelligence service	Python 3.12, FastAPI, Uvicorn
Scientific stack	NumPy, SciPy
Service authentication	PyJWT, verifying Supabase-issued tokens
Containerisation	Docker Desktop on WSL2 (Compose)
Python workspace	uv
JavaScript testing	Vitest, fast-check (property tests), Playwright (end-to-end)
Python testing and linting	pytest, ruff
Backend and hosting	Supabase (auth, database, storage), Vercel
Generation provider	OpenRouter, `deepseek/deepseek-v4-flash-0731`, temperature 0.3, reasoning off, 30 s timeout
Rubric-grading model	`deepseek/deepseek-v4-flash-0731` through `GRADING_MODEL`, the authoritative written-arm score
Feedback-copy model	`meta/muse-spark-1.3-contributor` through `FEEDBACK_MODEL`, reasoning effort low
Judgment client	Jev-1.13 hosted System One API through `OPENROUTER_JEV_API_KEY`, advisory shadow and loop-verdict judge
Code-execution sandbox	Piston sandbox profile on port 2000 for hidden-test grading
GPU inference sidecar	Qwen3-Embedding-0.6B and Qwen3-Reranker-0.6B on port 8200 in a separate compose project
Client code editor	CodeMirror with an advisory Pyodide check that never grades

Table 5.2: Datasets used in Phase II and their sources.

Dataset	Source	Role
Frozen corpus and harness golds	In-repository gold files: `eval_golds.json` with probe, written, and coding sets. ACCA text at 754 chunks with 768-dimensional sidecar vectors (edition needs verification)	Primary ground truth for the offline leg: generation, retrieval, mastery, and coding checks
Synthetic longitudinal histories	In-repository generator with four archetypes: steady, crammer, struggling, and returning	Whole-loop replay with known posture, and the post-fix replay evidence is versioned
OULAD	Public Open University dataset (version needs verification)	Calibration source for the Phase I generator
Real session logs	The candidate's own sessions	Reserved for future validation, outside the Phase II comparison

5.3 Model and Service Implementation

Each algorithm has one behavioral definition. The client and service hold separate implementations only where the client must compute a view locally. The client-side engines are written in TypeScript, and a Python mirror of each engine runs behind the intelligence service. A golden-fixture parity harness keeps the two implementations aligned.

The TypeScript engines emit reference inputs and outputs. A Python test replays each input and asserts numeric equality within a 1e-6 relative and absolute tolerance. The shared numeric constants, such as the BKT parameters, stay identical on both sides.

The split between client and service follows the architecture of Figure 4.4. Service-side code owns grounded generation, per-family grading arms, and the mastery endpoint. It also owns the guide stream with its gated reveal. It further owns the feedback-copy endpoint and the loop verdict with the Jev judge as a hosted-API client. The client keeps the engines it must read locally. Those are the roadmap bookings, the Gaussian-process projection, the mastery readers with the difficulty chip, and the verdict handoff.

The offline research harness imports the production Python engines as candidates. It never re-implements them, so an offline result measures the same engine code the service runs. Listing 5.1 shows the shipped BKT forward filter, which realizes Equation 2.1, and Table 5.3 lists the main packages.

Listing 5.1: The shipped BKT forward filter, which realizes Equation (2.1).

```
def bkt_forward(
    observations: list[MasteryObservation],
    params: BktParams = DEFAULT_PARAMS,
) -> tuple[list[float], list[float]]:
    """Run the forward filter over one skill's observation sequence.

    Returns ``(mastery_after, p_correct_before)`` parallel lists:
    ``p_correct_before[i]`` is the predicted probability of correctness for
    observation ``i`` computed from the belief before it, and
    ``mastery_after[i]`` is P(mastery) after processing observation ``i``.
    """

    mastery: list[float] = []
    p_correct: list[float] = []
    belief = params.p_init
    for obs in observations:
        p_correct.append((1.0 - params.p_slip) * belief + params.p_guess * (1.0 - belief)
    )
        if obs.correct:
            numerator = (1.0 - params.p_slip) * belief
            denominator = numerator + params.p_guess * (1.0 - belief)
        else:
            numerator = params.p_slip * belief
            denominator = numerator + (1.0 - params.p_guess) * (1.0 - belief)
        belief = numerator / denominator if denominator > 0 else belief
        belief += (1.0 - belief) * params.p_learn
        mastery.append(float(np.clip(belief, 0.0, 1.0)))
    return mastery, p_correct
```

Table 5.3: Main packages and their roles.

Package / module	Role
`packages/progress` (TS)	Client mastery engine with bkt-v1: projection and the difficulty chip
`packages/roadmap-engine` (TS)	Client booking and roadmap generation. The booking model is unchanged
`packages/py-progress` (Py)	Python mirror: Bayesian and enriched-shrinkage calibration, CUSUM, Gaussian process, bkt-v1
`packages/py-roadmap-engine` (Py)	Python mirror of the roadmap engine
`services/intelligence` (Py)	FastAPI service with a portless worker: generation, grading, mastery, guide, feedback, and the loop verdict
`services/embedder`	Qwen3 embedding and rerank sidecar in a separate compose project
Piston sandbox	External image for hidden-test code execution, started on demand
Research scripts	Offline harness and probes, importing the production engines

Code answers execute in the self-hosted Piston sandbox through the profile on port 2000. Hidden tests and reference solutions stay server-side. The client runs visible tests only as advice. No resource or egress bound for that sandbox is stated in this report (see Section 6.6).

Table 5.4 lists the Phase II service surface. Every route verifies the signed Supabase token before any model runs. Every call sends one `X-Request-ID`, and failures answer with typed service errors that fail closed.

Table 5.4: Phase II service surface: method, auth, contract, and errors.

Route	Contract
`POST /v1/assessments/generate`	One material, one family, one question. Tracked job. Rejects multi-family calls. Terminal states never retry
`POST .../attempts`	One answer per question. Never touches hidden tests or reference solutions
`POST /v1/guide/stream`	Server-sent events with monotonic sequence. Gated Reveal stays locked
`POST /v1/guide/reveal`	Requires confirmation plus the owned attempt bound to the question
`GET /v1/mastery, /recommendations`	Stateless per-skill estimate plus one-band advisory recommendation
`POST /v1/feedback/copy`	Drafts text for updated entries only. Static fallback. Never alters a grade
`POST /v1/loop/decision`	Judge proposes, code router disposes at evidence 0.5 and confidence 0.7
`POST /v1/retrieval/search`	Owner-scoped chunks. Optional rerank flag with top-k reorder

5.4 Evaluation Metrics and Reproducibility

The metrics used to score each loop stage are defined in Table 4.2 of Chapter 4. This section records how the project keeps them reproducible.

Three facts carry the reproducibility claim. First, every gate fails closed by default. The evaluation harness stays inert unless the caller passes `--activate`. The loop router maps a judge failure onto `hold` through a typed service error. The grading arms return a deterministic fail-closed outcome for unusable input and never fabricate a score.

Second, every call carries one request identifier from the browser to the service logs. The client sends `X-Request-ID` on each intelligence call. The service echoes it on each response, and every error log line carries it. One identifier therefore joins the frontend, the backend, and the end-to-end evidence.

Third, a reader can re-run both legs from versioned inputs and the documented services. The offline leg needs `eval_golds.json` with `eval_harness.py` and the documented Supabase, sidecar, and judge access. The longitudinal leg needs the synthetic generator with the four archetypes and the versioned post-fix replay evidence.

Table 5.5 records the hyperparameters a rerun must hold fixed. Table 5.6 records the test inventory behind the delivered code. There is no continuous-integration gate: verification is agent-run per the repository workflow, and coverage is not measured.

Table 5.5: Hyperparameters a rerun must hold fixed.

Parameter	**Value**
Chunking	About 400 tokens per chunk with about 30 tokens of overlap
Embedding	768 dimensions from the Qwen3 sidecar
Generation retrieval	Top 5 owner-scoped chunks per call
Endpoint search	Top 50 candidates before rerank
Generation model	Temperature 0.3, reasoning off, 30 s timeout, 4096 max output tokens
Repair	At most one repair. Hard format and citation gates
Attempt gate	K=5 over 3 fold hashes
BKT shipped params	p_{init} 0.15, p_{learn} 0.1, p_{slip} 0.05, p_{guess} 0.2
Loop gates	Evidence 0.5, confidence 0.7 (`LOOP_APPROVE_AT`)

Table 5.6: Test inventory behind the delivered code (counts collected 2026-10-23).

Suite	**Inventory**
Intelligence service	976 pytest tests collected across 57 test files
App and engines	65 vitest test files (unit plus provider tests)
Loop hook and feedback units	37 hook tests and 77 feedback tests passing in the repo suite
Browser end to end	33 Playwright spec files against the managed stack
Harness and probes	11 unit tests on the harness pure functions

5.5 The Phase II Application

The working application realizes the selected models. This section presents the Phase II additions. The Phase I screens are unchanged. Phase I delivered the planner with its session runtime, progress views, and roadmap lifecycle, and that substrate is unchanged.

This section follows five user-facing additions in order. Materials and ingestion (Section 5.5.1) is Phase I substrate. Assessment generation (FR-9) and taking and review (FR-10) come next. Practice and guidance (FR-11, FR-12, FR-14) and feedback with the loop verdict (FR-13, FR-15) are the remaining Phase II additions. The requirements behind them sit in Sections 3.3 and 3.4. Each subsection adds one stage to the flow diagram.

5.5.1 Materials and Ingestion

A material enters through the create page. The source grid offers plain text, PDF document, and web article side by side. The pipeline turns the upload into embedded chunks. The library card shows the material's current state until it is ready. A ready card offers practice instead of a status badge. A failed card keeps its badge with a retry action.

The detail page owns the rest of the lifecycle. It shows provenance and the outline, and it links onward to assessment and practice. Materials reach readiness in the library. Later subsections add the remaining stages to the chain in Figure 5.5.

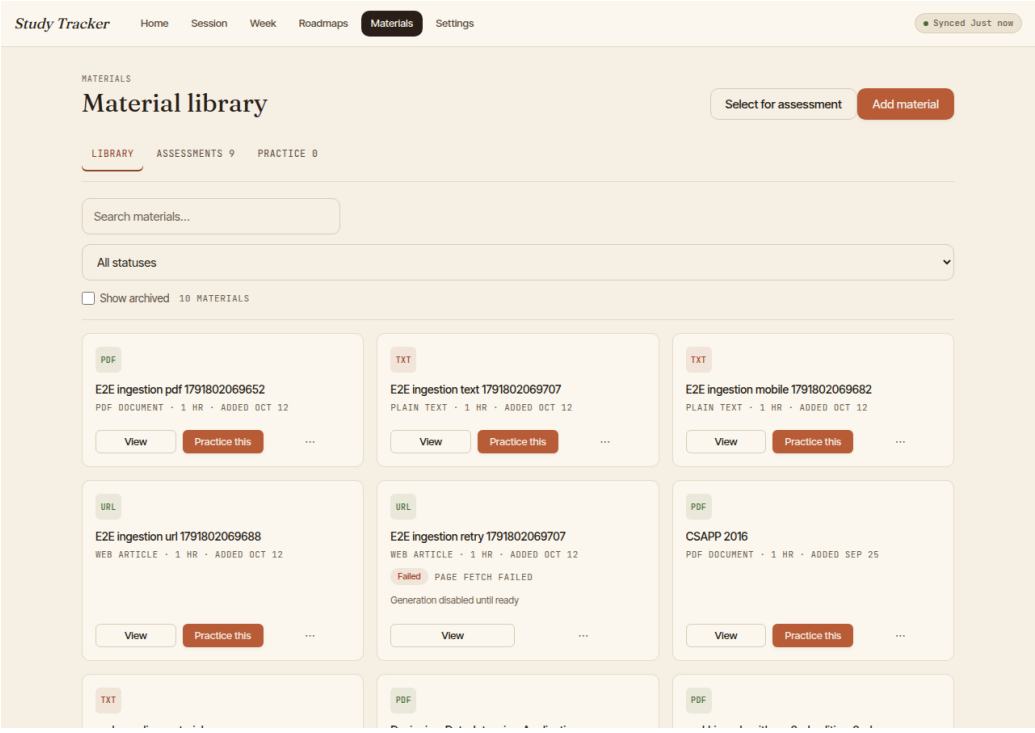


Figure 5.1: The material library: ready cards offer practice while the failed card keeps its badge with a retry action.

5.5.2 Assessment Generation

A generation run starts from the config page. The page binds one material, one family, and one scoped chapter before it requests a single question. The server gate admits exactly one family and one question per call. A mixed run alternates families across calls.

The assessment library lists every assessment with its family and status. Cache-first rows paint before revalidation completes. Generation runs as a tracked job. Only a run still marked generating retries in place. A failed run stays failed. Generation adds the second stage to the chain.

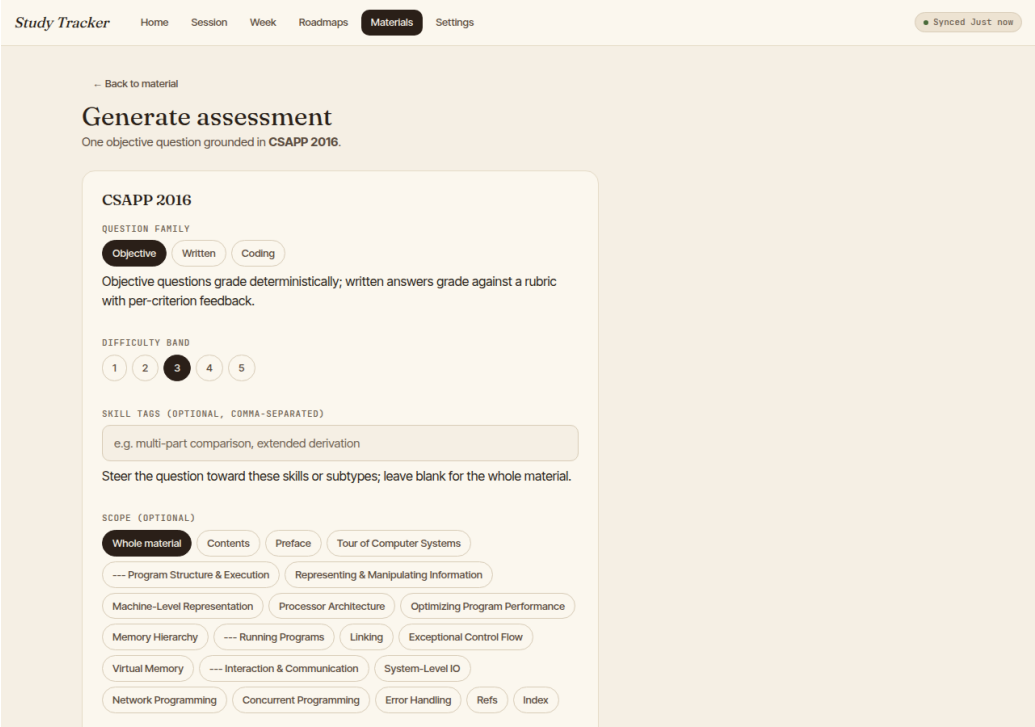


Figure 5.2: Assessment generation: one material, one family, and one scoped chapter per question.

5.5.3 Taking and Review

The taking screens collect one answer per question. They never touch hidden tests, reference solutions, or authored accepted values. Objective answers compare exactly against values authored at generation time. Written answers go to the rubric arm, and coding answers execute against hidden tests. Coding runs in an embedded editor with an advisory client check. The server grade stays final, and no client result enters the record.

The review module shows each attempt beside its grade with per-question detail. Taking and review add the third stage to the chain.

5.5.4 Practice and Guidance

The practice hub gathers runs across materials. Each run resolves its questions lazily from the loaded assessments and stores no copies. Pause means leave and return. Neither finish nor abandon

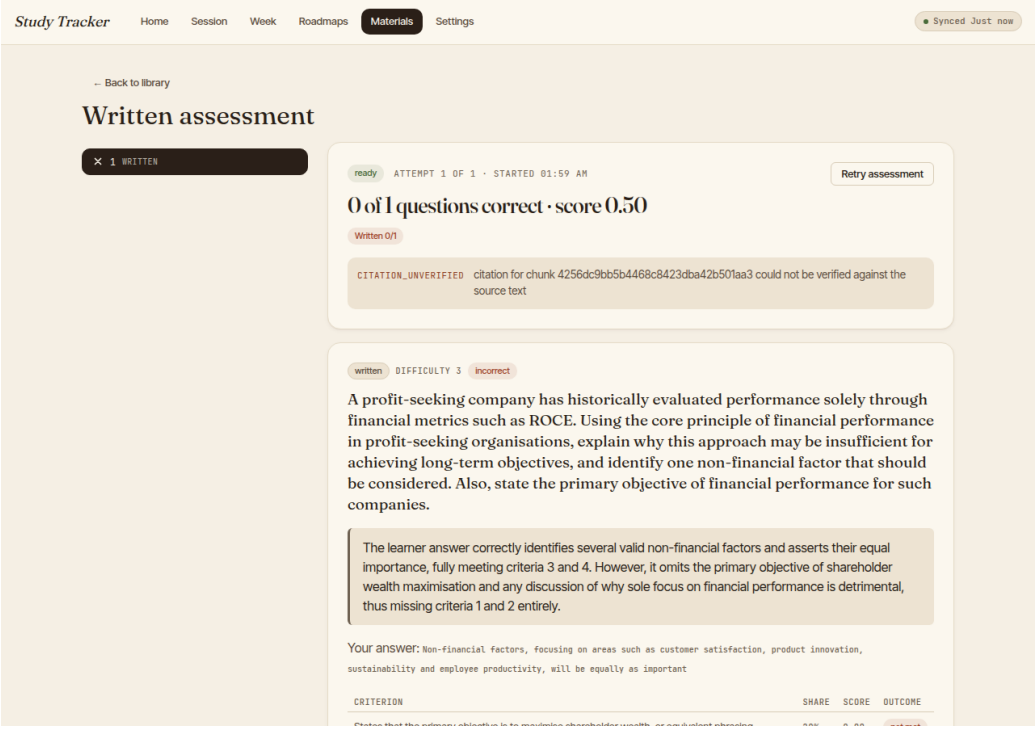


Figure 5.3: Taking and review: the server grade with per-criterion rubric feedback.

invents an observation.

The Socratic coach stays anchored to the page. It streams Nudge, Hint, and Targeted explanation against retrieved chunks and locks Reveal behind an explicit confirmation. The mastery reader beside it shows the current mastery estimate and its advisory, uncalibrated difficulty recommendation (Section 6.1). Practice and guidance add the fourth stage to the chain.

5.5.5 Feedback and the Closed Loop

The roadmap detail page carries an evidence ledger. The ledger shows one of four states, cold, updated, stale, or rebuilding, derived from the cached mastery and the latest copy. The feedback-copy endpoint drafts text for updated entries only. The static fallback covers every other state, and the copy never alters a grade.

The loop verdict proposes one choice over the week. Only `suggest_replan` passes its verdict pair to the replan screen. Confirming a replan mirrors the pair onto `RoadmapReplanned` and keeps the plan identity. `Keep` writes nothing. Figure 5.5 shows the complete chain.

These five additions complete the Phase II application. They are materials and ingestion, assessment generation, taking and review, practice and guidance, and the loop verdict with its replan. Chapter 6 reports the comparison behind them.

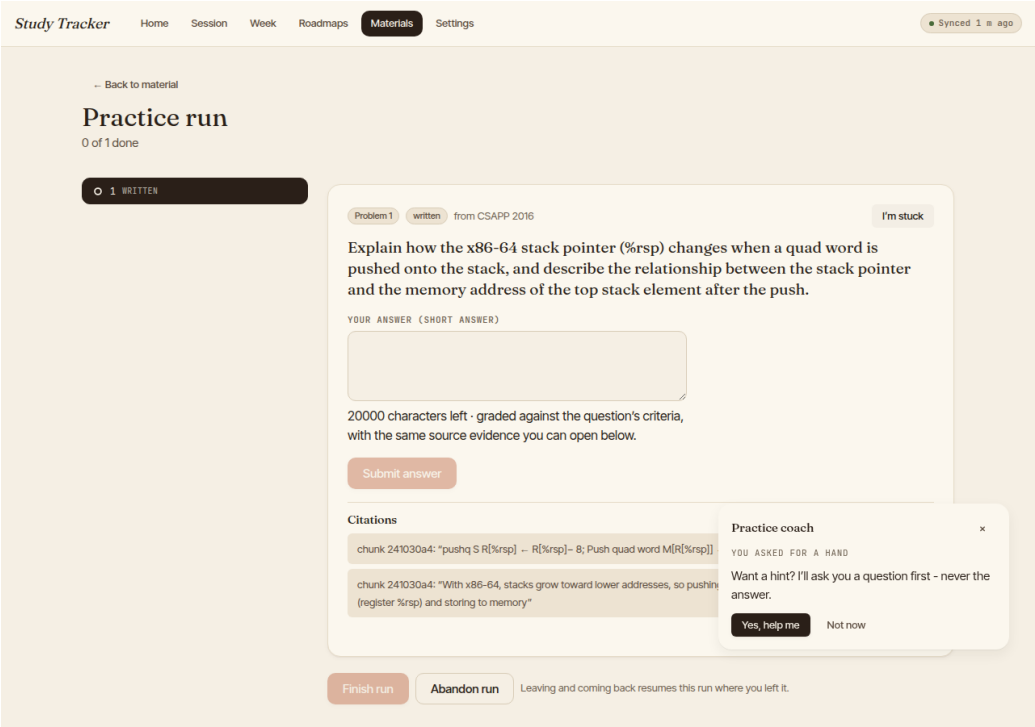


Figure 5.4: Practice run: the lazily resolved question with the Socratic coach anchored to the attempt.

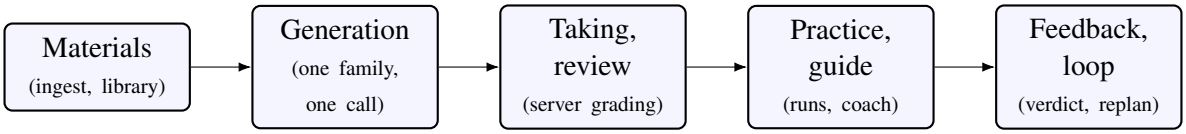


Figure 5.5: The complete Phase II application flow: materials reach readiness, generation binds one family per question, taking collects answers under server grading, practice resolves lazily with the coach, and the verdict completes the chain.

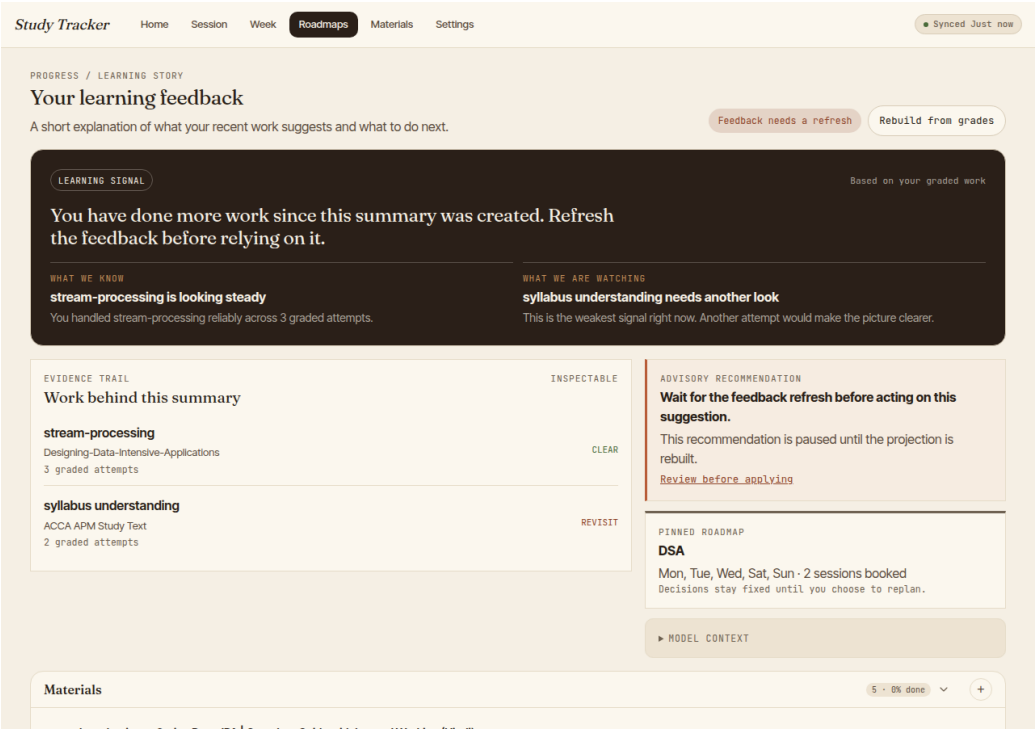


Figure 5.6: Feedback and the loop: the inspectable evidence trail with the advisory recommendation.

Chapter 6

Results and Discussion

This chapter reports the offline comparison for each Phase II loop stage and discusses what the numbers mean for the delivered system. All offline numbers run on the fixed artifacts described in Section 5.2, under the comparison design of Section 4.3. The metrics are those of Table 4.2. This chapter reports against them without redefining them.

Section 6.1 reports generation quality with the retrieval, generation, and tracing tables. Section 6.2 reports live verification as a deployment check. Section 6.3 reports the closed-loop verdict. Section 6.4 reports synthetic longitudinal validation. Section 6.5 discusses the results together.

6.1 Generation Quality

The offline leg scores components against fixed golds behind the fail-closed gate of Figure 4.3. Three tables carry the leg: retrieval with ranking, generation with its grading signals, and tracing with calibration. Reproducibility for both legs rests on Section 5.4. The harness stays inert unless activated, every call carries one request identifier, and both legs re-run from versioned files.

The harness report and the dated evidence files named in Section 5.4 name the exact inputs. The harness gate verdict for this run is FAIL. The breached gate, `difficultyMeanAbsBandError`, fires because difficulty calibration is unmeasured (Section 6.1).

Table 6.1 reports recall and MRR on the coherent 754-chunk ACCA split, the live endpoint row, and the DDIA second corpus. The harness dense and hybrid arms reproduce the recorded in-process baselines exactly, which validates the rerank arm beside them. Reranking lifts hybrid recall at 1 by 7 points (0.60 to 0.67) and recall at 3 by 10 points (0.80 to 0.90). MRR rises by 0.077 (0.720 to 0.797).

The live endpoint reproduces the plain RPC arm exactly (0.60/0.73/0.87/0.703) and its reranked row measures 0.70/0.90/0.93/0.813. The endpoint reranked row sits above the harness arm. The RPC candidate set differs slightly from the in-process copy, so per-question ranks move. The DDIA corpus moves the same direction with larger magnitude. Reranking lifts recall at 1 by 16 points (0.47 to 0.63) and recall at 3 by 17 points (0.73 to 0.90). MRR rises by 0.139 (0.660 to 0.799).

Wilson 95% intervals on n=30 span about 15 points: harness recall at 1 reads 0.42-0.75 and recall at 3 reads 0.56-0.86. Arm gaps inside that band are descriptive. On the paired endpoint rows the rerank gain is also descriptive. McNemar gives p=0.546 at rank 1 and p=0.131 at rank 3, so

neither clears 0.05. No multiplicity correction is applied anywhere in this chapter.

Cross-corpus absolutes are not comparable because the two gold sets were authored independently. Only the arm deltas transfer. The retired 788-split headline (recall at 1 of 0.77, recall at 3 of 0.97, MRR of 0.858) is the frozen-split precedent only. Two rejected arms stay on record: title-prefix indexing shows no gain, and int8 quantization collapses to 0.10 across recalls at 2.45 times speedup.

Table 6.1: Retrieval: recall and MRR by arm. ACCA panels share the 30-question harness; the two endpoint rows are the live deployed path (plain RPC plus reranked); DDIA is the second corpus.

Corpus and arm	**r@1**	**r@3**	**r@5**	**MRR**
ACCA 754-chunk harness				
dense	0.60	0.77	0.87	0.706
hybrid	0.60	0.80	0.87	0.720
hybrid_rerank	0.67	0.90	0.93	0.797
Live endpoint, same corpus				
plain (RPC)	0.60	0.73	0.87	0.703
reranked	0.70	0.90	0.93	0.813
DDIA 1094-chunk second corpus				
dense	0.40	0.73	0.83	0.593
hybrid	0.47	0.73	0.83	0.660
hybrid_rerank	0.63	0.90	0.97	0.799

Generation headlines the delivered reasoning-off arm (S:off, n=30). Schema validity reads 0.9667 (29 of 30) and gold support reads 0.7241 (21 of 29 valid citations). Latency p95 is 10.5 s at cost $0.00561. Table 6.2 carries the six-tier sweep aggregate beside it. The sweep (S-layer, n=180) reads schema validity 0.9611, citation validity 0.9133, gold support 0.7803, near-duplicate rate 0.0044, cost $0.061289. The generation telemetry behind the sweep counts 361 requests: 281 ok, 37 partial, 35 malformed output, 8 provider errors.

Groundedness measures 0.8667 (26 of 30 judged answerable, Wilson 95% 0.70-0.95). The 30 judgments cover six distinct questions (S:1 to S:6 with three to six repeats each). The rate describes six clusters rather than 30 independent items. Both the generator and the judge run `deepseek-v4-flash-0731` with reasoning off. The score is self-judged, and no independent-judge rerun or human audit exists.

The two numbers measure different things. The judge scores whether the cited chunk supports the answer, while gold support scores whether the citation contains the gold string. Schema validity is 0.9611 and citation validity 0.9133, with a near-duplicate rate of 0.0044 at a cost of $0.061289.

The coding arm refuses material it cannot ground, so derivability on the golds is 0.0 on the pre-top-up run. The fresh live pull derives at 0.5349 over 43 judged attempts (Wilson 95% 0.39-0.68). Over the full 62 attempts the rate is 0.371 (23 derivable, 20 refusals, 19 failed for other reasons). The rate does not clear the 0.5 gate. The 0.5349 clears it only on the judged pair.

Gold agreement reads 3 of 3. The gold counts are 30 objective, 6 written, and 4 coding (3 derivable plus 1 not), with attempt gate K=5 over 3 fold hashes. The written arm kept its rubric cutoff 0.6 and margin 0.10. The train split favored 0.7, but test agreement fell to 0.5 on a met class of two pairs, so the 0.6 prior shipped.

Table 6.2: Generation: delivered S:off arm, S-layer sweep aggregate, groundedness judge, and coding pair. Judge and citation metrics are distinct by construction.

Metric	Value
Delivered arm S:off (n=30)	
Schema valid (S:off, 29 of 30)	0.9667
Gold support (S:off, 21 of 29)	0.7241
Cost and p95 latency (S:off)	$0.00561, 10.5 s
Six-tier sweep aggregate (n=180)	
Schema valid	0.9611
Citation valid	0.9133
Gold support (citation-contains-gold)	0.7803
Near-duplicate rate	0.0044
Cost (USD)	0.061289
Telemetry (ok, partial, malformed, provider error)	281, 37, 35, 8
Groundedness, LLM-judged (26 of 30 over six questions)	0.8667
Coding derivable on golds (pre-top-up run)	0.0
Coding derivable live (23 of 43 judged)	0.5349
Coding derivable live (23 of 62 attempted)	0.371

Tracing accuracy across three populations sits in Table 6.3. The bake-off fixed the shipped parameters (p_init 0.15, p_learn 0.1, p_slip 0.05, p_guess 0.2) at mean AUC 0.785 with ECE 0.057. The live harness over 282 observations across 139 skill sequences gives bkt-v1 AUC 0.7397 with ECE 0.1401 against running-proportion 0.7368 with 0.2679. The external pyBKT accoding anchor reads 0.602 to 0.611 under gate `masteryMinAuc` 0.6, with ECE 0.272 marked not reportable.

The reading is parity on ranking and a gain on calibration. bkt-v1 matches its baseline on ranking while the calibration error nearly halves on live attempts. The Chapter 4 bake-off pair (0.888 against 0.889) says the same on its cut. The evidence files carry point estimates only, so no interval accompanies AUC or ECE and the parity reading stays descriptive.

Difficulty calibration is recorded as unmeasured. The current pool holds 23 observations on a single question in one band, so the band error is null by construction. The adaptive chip stays uncalibrated-advisory. The update itself is Equation 2.1 with the shipped filter in Listing 5.1.

Table 6.3: Knowledge tracing: AUC and calibration across three populations. Parity on ranking; calibration nearly halves on live attempts.

Population	AUC	ECE / note
Synthetic bake-off (shipped params)	0.785	0.057
Live harness bkt-v1 (282 obs, 139 seqs)	0.7397	0.1401
Live harness running-proportion	0.7368	0.2679
External pyBKT accoding anchor	0.602-0.611	gate 0.6, ECE 0.272 not reportable

6.2 Live Verification

The live matrix confirms deployment and never promotes a candidate, per Section 4.3. Objective runs take 2 of 2, written runs grade 4 of 4 with real rubric grades, and coding runs grade 3 of 3. Guide runs pass 2 of 2, practice runs pass 2 of 2, and feedback copy passes 3 of 3. Mastery passes 2 of 2 and review passes 4 of 4. Ingestion passes 5 with 2 skipped.

Every run reports zero console and page errors. AC1, AC3, and AC4 pass with AC2 passing with skips. Each call joins frontend to service logs on one request identifier per Section 5.4.

At scale, the full-book E2E runs through the sidecar. The 572-page fixture (about 23 MB) moves from upload to created at 13.5 s and extracting at 15 s. Embedding takes 48 s and ready takes 101 s. The start-to-ready wall is about 88 s at 10 s poll granularity. The same fixture yields the durable 754-chunk baseline with 0 NULL embeddings. The material was deleted after the run with the card-gone state verified, so the shared account sits as found.

6.3 Closed-Loop Decision

Table 6.4 is this chapter's verdict evidence: the post-fix replay converts 4 of 4 `suggest_replan` verdicts into verdict-linked replans. Confidence measures 1.0 with sufficiency from 0.66 to 0.77 across the four archetypes. Every replan extends the deadline and mirrors the exact verdict pair onto `RoadmapReplanned`. The span runs 32 days for all four from the durable material-scoped `SessionLogged.payload.date`. The pre-fix contrast is 4 of 4 `hold` at confidence 0.18 to 0.34 on spanless neutral inputs.

The input fix opened these states: confidence 1.0 would also clear the old 0.9 gate. The gate moved from 0.9 to 0.7. The 0.8 candidate was rejected after a live suggest_replan sample at 0.79 fell inside the 0.79-0.85 band.

Table 6.4: Loop verdict: post-fix replay across four archetypes. Sufficiency 0.66 to 0.77 is the cross-user range; per-archetype decidedAt values sit in the versioned JSON beside the replay evidence.

Archetype	Verdict	Conf	Suff	Option	Converted
steady	suggest_replan	1.0	0.66-0.77	extend-deadline	yes
crammer	suggest_replan	1.0	0.66-0.77	extend-deadline	yes
struggling	suggest_replan	1.0	0.66-0.77	extend-deadline	yes
returning	suggest_replan	1.0	0.66-0.77	extend-deadline	yes

6.4 Synthetic Longitudinal Validation

The longitudinal leg replays four synthetic archetypes with known posture: steady, crammer, struggling, and returning. Development histories stay separate from evaluation histories, so a conversion reflects behavior instead of fitting. The 24-cell adherence emission and the per-archetype decidedAt values sit in Appendix A. Twenty weekly values are observed. Week 5 is not observed for any user. This section reports the headline with its caveat.

Next-week adherence is not observed for any of the four in the post-replan extension week. Capture ran one day before the deadline, so no post-replan sessions could be logged. The 0.000 in Table A.1 marks an unobserved week, not a measured zero and not a learning result.

The 12 mastery rows sit at about 0.1067 throughout after the replay, below the prior of 0.15 by construction. The seeded scripts answer about 90 percent wrong because they prioritize grading-path coverage over learner success. Those rows measure the scripts and are never cited as tracing validation. The tracing evidence lives in Table 6.3.

Each of the 4 verdicts joins its `RoadmapReplanned` by the exact verdict pair with createdAt at or after decidedAt. A seed-shape finding is recorded with the evidence. The seed writes one timestamp with a zone offset while the booking reference uses milliseconds. The strict-equality join silently mistraces until emission canonicalizes the instant. The seed-shape defect was recorded with the evidence. No product code changed.

The dip-window replay pins the browser clock to the week-3 trough for every archetype. All four hold at confidence 0.28 to 0.35 with sufficiency 0.66 to 0.77 and convert nothing, because `Keep` writes nothing. A September-12 pin for the struggling archetype flips the choice to `suggest_replan` at 0.61 to 0.69 but the gate holds it. The gradient matches the design: trough holds on the merits, week 4 proposes below the gate, and the rebound eve converts at 1.0.

The replay drove verdicts only and produced no new graded attempts, so the difficulty pool is unchanged. Cloud replan counts stayed at the recorded per-user values (crammer 1, returning 2, steady 1, struggling 2), with no cloud history rewritten. Four caveats travel with this leg. The clock pins to 2026-10-14, and capture covers the rebound era rather than the week-3 dip. The onboarding gate is local only, and cold restore duplicates locally while extracting from cloud only.

6.5 Discussion

Read together, the results move the loop from logged time to graded evidence. Tracing holds ranking parity while the calibration error nearly halves on live attempts, because the per-skill updater tracks the graded attempts the baseline averages away. Reranking lifts recall on both corpora, with the larger DDIA gain coming from the noisier candidate set. Coding refuses ungroundable material instead of inventing a question, and the live pull clears the 0.5 gate only on the judged pair.

Difficulty calibration is unmeasured because the pool holds one question in one band. Next-week adherence is unobserved because capture ran one day before the deadline. Two results are comparative gains: retrieval ranking and the 4 of 4 verdict conversion with its pre-fix contrast. Two are descriptive: grounded generation at 0.8667 with near-duplicates at 0.0044, and the full-book path reaching ready in about 88 s.

One refinement stays validated but unshipped from Phase I: the split-conformal interval correction. It does not enter the delivered behavior and returns in Chapter 7 as future work.

Transfer holds in the sense the design needs. Dense and hybrid reproduce their baselines exactly on the rerun, which validates the rerank arm. The DDIA corpus repeats the ACCA direction at larger magnitude.

The scope is a single learner with synthetic histories in place of a classroom study. No fixed-plan control arm ran, so the 4-of-4 conversion has no efficacy comparison. One difficulty question is eligible, with the four replay caveats of Section 6.4. Figure 4.4 shows where each stage from Table 4.1 now runs.

6.6 Threats to Validity

Construct validity is limited on two fronts. The groundedness score is self-judged: the generator and the judge share the same model with reasoning off, and no independent judge or human audit exists. The longitudinal scripts prioritize grading-path coverage over learner success and answer about 90 percent wrong, so the mastery rows measure the scripts rather than tracing.

Internal validity rests on synthetic histories with no fixed-plan control, so the verdict conversion cannot claim efficacy on learning. Week 5 is unobserved rather than measured. The 0.5 coding gate clears only on the judged pair. The full-denominator rate of 0.371 does not clear it. The harness gate verdict for this run is FAIL on the unmeasured difficulty band.

External validity is narrow: one learner, one frozen ACCA textbook split plus the DDIA second corpus, and pinned model versions with no drift monitoring. Retrieval and generation numbers need not transfer to other materials or models.

An ethics and security paragraph travels with these limits. Learner material sent to OpenRouter for generation, grading, and judging travels to a third party, and the report states no retention or redaction contract. A classroom deployment must add one. Piston executes hidden tests server-side through the profile on port 2000, but the report states no resource or egress bound for that sandbox. Adversarial material that injects instructions into generation or grading is untested. No learner usability evidence exists: every live check is a deployment pass by the developer.

6.7 Summary

Read together, the results validate the integration rather than the learning. Reranking lifts recall on both corpora, but the paired test cannot separate the gain from noise at n=30. The delivered S:off arm generates valid questions cheaply, and groundedness reads 0.8667 over six self-judged clusters. Coding clears its gate only on the judged pair. Tracing holds ranking parity while nearly halving calibration error.

The loop converts 4 of 4 verdicts into verdict-linked replans with no fixed-plan control, and next-week adherence is unobserved. The system is a working integration of graded evidence into planning, with efficacy on learning deferred.

Chapter 7

Conclusion and Future Work

This chapter closes the report. Section 7.1 concludes what the two phases delivered. Section 7.2 describes future work.

7.1 Conclusion

Phase I built an open-loop adaptive planner for a single self-directed learner. Phase I combined a Bayesian pace estimate and enriched shrinkage with CUSUM shift detection. It projected completion with a Gaussian process and a cold-start composite, then regenerated the plan with capacity bookings. Evaluation reported nulls on retrospective recovery error and change detection, and wins on prospective calibration and scheduling.

Phase II closed the loop with assessment-based verification of graded evidence. The project's main contribution is the verified-learning closed loop, which extends the predict-then-optimize idea [3] from predicted grades to graded evidence from the learner's own answers.

The closest prior is the Cognitive Tutor line [18, 17]. It combines step tracing, knowledge tracing, and mastery learning, reviewed at near-human tutoring effects [16]. This project narrows that architecture to one self-directed learner without a classroom. It uses generated grounded items instead of authored curricula, per-family server grades instead of step tracing, and a verdict-driven replan instead of a teacher. Logged time is self-reported, so it cannot evidence understanding. Graded responses come from a server-authoritative arm, with the guidance ladder gating the answer, though help-seeking can still be gamed [11].

Table 6.1 shows retrieval gains on both corpora, with the dense and hybrid arms reproducing their recorded baselines. Reranking lifts hybrid recall at 1 from 0.60 to 0.67 on the 754-chunk ACCA split and from 0.47 to 0.63 on the DDIA corpus. MRR rises by 0.077 on ACCA and by 0.139 on DDIA, while the live endpoint is 0.813 reranked. On the retired 788 split, title-prefix indexing shows no gain, and int8 quantization collapses to 0.10. The retired 788-split headline stays as frozen-split precedent only.

Grounded generation measures 0.8667 on the judge against 0.7803 gold support, with schema validity at 0.9611 and citation validity at 0.9133. The two metrics measure different things. The coding arm refuses material it cannot ground, so derivability on the golds reads 0.0 on the pre-top-up run. A fresh live pull derives at 0.5349 over 43 judged attempts (0.371 over all 62 attempts). The 0.5 gate clears only on the judged pair.

Tracing holds at parity on ranking while calibration nearly halves on live attempts. bkt-v1 measures 0.7397 against 0.7368 for the running proportion over 282 observations, with ECE at 0.1401 against 0.2679. The shipped parameters hold mean AUC at 0.785 with ECE at 0.057, and the external anchor gives 0.602 to 0.611. Difficulty calibration stays unmeasured, with 23 observations on a single question in one band.

Live verification confirms deployment only, with objective runs at 2 of 2, written grades at 4 of 4, and coding grades at 3 of 3. The full-book path moves from created to ready in about 88 s (101 s from upload start) for the 572-page fixture.

The loop verdict converts 4 of 4 `suggest_replan` verdicts into verdict-linked replans, as Table 6.4 records. Confidence measures 1.0 with sufficiency from 0.66 to 0.77, and all four synthetic replays extended the deadline over a 32-day span. The pre-fix contrast is 4 of 4 hold at 0.18 to 0.34, and the gate moved from 0.9 to 0.7.

Synthetic longitudinal replay leaves next-week adherence unobserved, since capture ended one day before the deadline, and the 24-cell adherence emission sits in Appendix A. The dip-window replay holds 4 of 4 at 0.28 to 0.35. The separate Oct-14 rebound-eve run converts at 1.0. The only validated refinement to a shipped component that stays unshipped is the split-conformal correction.

Together, the loop demonstrates graded evidence steering a plan on synthetic histories with no fixed-plan control. Retrieval grounds the questions, the rubric and sandbox arms grade them, and the verdict converts the grades into a replan. Learning gain stays undemonstrated: no run measured it, and it remains future work.

7.2 Future Work

Section 5.4 records the versioned artifacts that make both legs reproducible. The appendix carries the full adherence and verdict tables. Two near-term lines follow. The first is to ship the validated refinement: the split-conformal correction of the projection interval, which Section 6.5 leaves as the deferred calibration fix. The second is difficulty calibration with two or more bands, so the adaptive chip moves from advisory to measured. It needs more eligible questions than the single band available here, and a thicker met-class than the current two pairs.

Two longer-term lines close the report. Longitudinal validation on real data compares closed-loop planning against the fixed-plan baseline on adherence and on a pre-registered assessment of learning gain. A single-learner study over one term is the next step beyond synthetic histories, and it answers the rebound-era capture caveat of Section 6.4. The smallest usability gate before that study is an expert heuristic review or a 3-to-5 learner pilot. No usability evidence exists yet. Classroom scope extends the single-learner loop to groups without losing per-learner mastery.

7.3 Limitations

The limits of Section 6.6 stand as the closing word. The evidence is synthetic and single-learner. The groundedness judge grades its own generator over six questions. The coding gate clears only on the judged pair, difficulty calibration is unmeasured, and week 5 is unobserved.

No fixed-plan control ran, so no efficacy comparison exists. Learner material travels to a third party with no stated retention contract, the sandbox runs without a stated resource bound, and

prompt injection is untested. The contribution is a working integration of graded evidence into planning, not measured learning gain.

Appendix A

Supplementary Tables

This appendix collects the supplementary tables and figures referenced from Chapter 6. Each item repeats its in-text number so the reader can move between the discussion and the full table without searching.

A.1 Adherence Emission

Table A.1 lists the adherence emission from Section 6.4. Twenty weekly values are observed across four users and weeks 0 to 4, plus a week 5 that is not observed for any user. Planned time reads 390 minutes for weeks 0 to 4 and 360 minutes for week 5. Week 5 carries no sessions because capture ended one day before the deadline (Section 6.4).

Table A.1: Adherence emission: actual over planned minutes by user and week. Week 5 is not observed (n.o.) for any user.

User	w0	w1	w2	w3	w4	w5
steady	0.692	0.921	0.879	0.954	0.933	n.o.
crammer	0.215	0.138	0.385	0.254	0.923	n.o.
struggling	0.323	0.169	0.385	0.144	0.554	n.o.
returning	0.656	0.000	0.000	0.700	0.900	n.o.

A.2 Verdict Instants

Table A.2 gives the per-user verdict instants from Section 6.4. Each row joins its RoadmapReplanned by the exact verdict pair. All four converted. The 12 mastery rows are excluded here by design. They measure the seeded scripts at about 0.1067 throughout. The versioned JSON beside the replay evidence holds them.

Table A.2: Loop verdict instants: per-user decidedAt values.

User	Verdict	decidedAt	Converted
steady	suggest_replan	2026-10-14T12:00:11.235Z	yes
crammer	suggest_replan	2026-10-14T12:00:07.156Z	yes
struggling	suggest_replan	2026-10-14T12:00:07.606Z	yes
returning	suggest_replan	2026-10-14T12:00:07.515Z	yes

A.3 Second-Corpus Questions and Endpoint Pointers

Table A.3 lists the 30 DDIA gold questions from Section 6.1. The driver verified each snippet in at least one chunk before the run. Per-question ranks for the endpoint and harness arms stay in their versioned files:

- `research/doc/2026-10-22-endpoint-retrieval.json`
- `research/doc/2026-10-22-reranked-754.json`
- `research/doc/2026-10-22-second-corpus-ddia.json`

The DDIA gold stems sit in `research/doc/2026-10-22-ddia-questions.json` beside the run files.

A.4 Prompts and Reproduction

The offline groundedness judge runs the prompts below verbatim. The source is the offline harness from Section 5.4.

```
System: You audit whether a generated exam question is answerable purely from the source
    chunks it cites. Answer strictly from the chunks; do not use outside knowledge.
    Respond only with the required JSON object.
User: Question: {question} Cited chunks: {chunks} Is the question answerable purely from
    these chunks?
```

The schema requires one boolean `answerable` with a `reason` string. Production prompts live beside the code they serve. Generation shapes live in the generation `prompts.py` module (template version `v1`, one family per call). The written rubric contract lives in the grading `rubric_grader.py` module. The repair gates live in the generation `validation.py` module.

Both legs rerun from versioned inputs. Offline, from `services/intelligence/`:

```
uv run --package intelligence python scripts/eval_harness.py --activate
```

Append `--groundedness-judge --limit 2` for the dry run and `--verify-golds` to check the gold registry. The longitudinal leg replays the four synthetic archetypes through the verdict endpoint and extracts the adherence, mastery, and verdict tables with the synth extractor. The seed inputs live in the app dev driver and the post-fix outputs are versioned beside the dated synth-extract evidence files.

Table A.3: DDIA second-corpus gold set, questions 1 to 15. Snippets are verbatim from the evidence file.

Q	Label	Question	Answer snippet
1	reliability meaning	What does it mean for a data system to be reliable?	The system should continue to work correctly (performing the correct function at the desired level of performance) even in the face of adversity
2	fault vs failure	What is the difference between a fault and a failure?	Note that a fault is not the same as a failure
3	scalability meaning	How does the book define scalability?	Scalability is the term we use to describe a system's ability to cope with increased load
4	maintainability operability	What does operability mean as an aspect of maintainability?	Make it easy for operations teams to keep the system running smoothly
5	document schema flexibility	What does schema flexibility mean in the document model?	Most document databases, and the JSON support in relational databases, do not enforce any schema on the data in documents
6	relational roots	What are the roots of the relational model?	proposed by Edgar Codd in 1970
7	NoSQL drivers	What drove the adoption of NoSQL databases?	need for greater scalability than relational databases can easily achieve
8	declarative queries	Why are declarative query languages preferred over imperative APIs?	declarative query languages like SQL turned out to be much better than imperative query APIs
9	SSTable merge	What is an advantage of SSTables over log segments with hash indexes?	Merging segments is simple and efficient, even if the files are bigger than the available memory
10	B-tree vs LSM reads	How do B-trees and LSM-trees compare on read performance?	LSM-trees are typically faster for writes, whereas B-trees are
11	secondary indexes	How common are secondary indexes in relational databases?	It is also very common to have secondary indexes
12	replication leaders	How does leader-based replication work?	One of the replicas is designated the leader
13	read-your-writes	What guarantee prevents reading stale data after your own write?	read-after-write consistency, also known as read-your-writes consistency
14	multi-leader conflicts	What is the biggest problem with multi-leader replication?	The biggest problem with multi-leader replication is that write conflicts can occur
15	quorum reads	When can quorum reads generally expect the most recent value?	If you have n replicas, and you choose w and r such that w + r > n

Table A.3: DDIA second-corpus gold set, questions 16 to 30.

Q	Label	Question	Answer snippet
16	skew batch	How does skew affect batch jobs?	due to skew (data not being spread evenly across worker processes) and needing to wait for the slowest task to complete
17	consistent hashing	What is consistent hashing?	Consistent hashing, as defined by Karger et al. [7], is a way of evenly distributing load
18	ACID coined	Where does the acronym ACID come from?	It was coined in 1983 by Theo Härder and Andreas Reuter
19	read committed	What is the most basic transaction isolation level?	The most basic level of transaction isolation is
20	snapshot isolation term	Which isolation level is named in the chapter on weak isolation?	snapshot isolation
21	serializable strongest	How strong is serializable isolation?	Serializable isolation is usually regarded as the strongest isolation level
22	2PC coordinator	What extra element appears in distributed transactions compared to single-node ones?	a coordinator (also known as transaction manager)
23	FLP result	What does the FLP result prove about consensus?	You may have heard about the FLP result
24	CAP consistency meaning	What does consistency mean in the CAP theorem?	the word consistency is used to mean
25	linearizability illusion	What illusion does linearizability give?	could give the illusion that there is only one replica
26	version vectors	What technique orders events across leaders?	a technique called version vectors can be used
27	batch definition	What is a batch processing system?	A batch processing system takes a large amount of input data, runs a job to
28	event vs processing time	What is the difference between event time and processing time?	Event time versus processing time
29	Kreps lambda	Who questioned the lambda architecture?	Jay Kreps: “Questioning the Lambda Architecture
30	exactly-once meaning	What does exactly-once mean for stream processing?	the final effect is the same as if no faults had occurred

Bibliography

[1] Chen, A. M., Palacci, A., Vélez, N., Hawkins, R. D., Gershman, S. J., "A Hierarchical Bayesian Model of Adaptive Teaching," *Cognitive Science*, vol. 48, no. 7, Art. e13477, 2024, doi:10.1111/cogs.13477.

[2] Alhazbi, S., et al., "Using Learning Analytics to Measure Self-Regulated Learning: A Systematic Review," *Journal of Computer Assisted Learning*, vol. 40, no. 4, pp. 1658–1674, 2024, doi:10.1111/jcal.12982.

[3] Islam, M., et al., "Predict Students' Performance and Optimize Preparatory Leave," *Proc. 27th Int. Conf. on Computer and Information Technology (ICCIT)*, IEEE, pp. 728–733, 2024, doi:10.1109/iccit64611.2024.11021881.

[4] Zanellati, A., Di Mitri, D., Gabbrielli, M., Levrini, O., "Hybrid Models for Knowledge Tracing: A Systematic Review," *IEEE Transactions on Learning Technologies*, vol. 17, pp. 1021–1036, 2024.

[5] Mai, N. T., Cao, W., Liu, W., "Interpretable Knowledge Tracing via Transformer–Bayesian Hybrid Networks," *Applied Sciences*, vol. 15, Art. 9605, 2025, doi:10.3390/app15179605.

[6] Chan, K. W., et al., "Automatic Item Generation in Various STEM Subjects Using Large Language Model Prompting," *Computers and Education: Artificial Intelligence*, vol. 8, Art. 100344, 2024, doi:10.1016/j.caeai.2024.100344.

[7] Shintani, S. A., "Self-hosted Lecture-to-Quiz: Local LLM MCQ Generation with Deterministic Quality Control," arXiv:2603.08729 [cs.CY], 2026 (unrefereed preprint).

[8] Moore, S., Schmucker, R., Mitchell, T., Stamper, J., "Automated Generation and Tagging of Knowledge Components from Multiple-Choice Questions," *Proc. 11th ACM Conf. on Learning @ Scale (L@S '24)*, Atlanta, GA, USA, 2024, doi:10.1145/3657604.3662030.

[9] Ta, N. B. D., Chai, Y. M., "Automatic Grading of Short Answers Using Large Language Models in Software Engineering Courses," *Proc. 2024 IEEE Global Engineering Education Conf. (EDUCON)*, Kos Island, Greece, pp. 1–10, 2024, doi:10.1109/educon60312.2024.10578839.

[10] Uzun, Y., Suraworachet, W., Zhou, Q., Gauthier, A., Cukurova, M., "Engagement with Analytics Feedback and Its Relationship to Self-Regulated Learning Competence and Course Performance," *International Journal of Educational Technology in Higher Education*, vol. 22, Art. 17, 2025, doi:10.1186/s41239-025-00515-3.

[11] Zhang, J., et al., "Understanding Gaming the System by Analyzing Self-Regulated Learning in Think-Aloud Protocols," *Proc. 16th Int. Learning Analytics and Knowledge Conf. (LAK)*, ACM, 2026, doi:10.1145/3785022.3785025.

[12] Corbett, A. T., Anderson, J. R., "Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge," *User Modeling and User-Adapted Interaction*, vol. 4, pp. 253–278, 1995, doi:10.1007/BF01099821.

[13] Page, E. S., "Continuous Inspection Schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954, doi:10.1093/biomet/41.1-2.100.

[14] Rasmussen, C. E., Williams, C. K. I., *Gaussian Processes for Machine Learning*, MIT Press, 2006.

[15] Vovk, V., Gammerman, A., Shafer, G., *Algorithmic Learning in a Random World*, Springer, 2005.

[16] VanLehn, K., "The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems," *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011, doi:10.1080/00461520.2011.611369.

[17] Koedinger, K. R., Aleven, V., "Exploring the Assistance Dilemma in Experiments with Cognitive Tutors," *Educational Psychology Review*, vol. 19, no. 3, pp. 239–264, 2007, doi:10.1007/s10648-007-9049-0.

[18] Anderson, J. R., Corbett, A. T., Koedinger, K. R., Pelletier, R., "Cognitive Tutors: Lessons Learned," *Journal of the Learning Sciences*, vol. 4, no. 2, pp. 167–207, 1995, doi:10.1207/s15327809jls0402_2.

[19] Cepeda, N. J., Pashler, H., Vul, E., Wixted, J. T., Rohrer, D., "Distributed Practice in Verbal Recall Tasks: A Review and Quantitative Synthesis," *Psychological Bulletin*, vol. 132, no. 3, pp. 354–380, 2006, doi:10.1037/0033-2909.132.3.354.

[20] Settles, B., Meeder, B., "A Trainable Spaced Repetition Model for Language Learning," *Proc. 54th Annual Meeting of the Association for Computational Linguistics (Vol. 1: Long Papers)*, pp. 1848–1858, 2016, doi:10.18653/v1/P16-1174.

[21] Zheng, L., et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," *Advances in Neural Information Processing Systems 36 (Datasets and Benchmarks Track)*, pp. 46595–46623, 2023.

[22] Embretson, S. E., Reise, S. P., *Item Response Theory for Psychologists*, Erlbaum, 2000.

[23] Hattie, J., Timperley, H., "The Power of Feedback," *Review of Educational Research*, vol. 77, no. 1, pp. 81–112, 2007, doi:10.3102/003465430298487.

[24] Chi, M. T. H., Wylie, R., "The ICAP Framework: Linking Cognitive Engagement to Active Learning Outcomes," *Educational Psychologist*, vol. 49, no. 4, pp. 219–243, 2014, doi:10.1080/00461520.2014.965823.

[25] Piech, C., et al., "Deep Knowledge Tracing," *Advances in Neural Information Processing Systems 28*, 2015, arXiv:1506.05908.

[26] Niculescu-Mizil, A., Caruana, R., "Predicting Good Probabilities with Supervised Learning," *Proc. 22nd Int. Conf. on Machine Learning (ICML)*, pp. 625–632, 2005, doi:10.1145/1102351.1102464.

[27] Bloom, B. S., "The 2 Sigma Problem: The Search for Methods of Group Instruction as Effective as One-to-One Tutoring," *Educational Researcher*, vol. 13, no. 6, pp. 4–16, 1984, doi:10.3102/0013189X013006004.

[28] Shute, V. J., "Focus on Formative Feedback," *Review of Educational Research*, vol. 78, no. 1, pp. 153–189, 2008, doi:10.3102/0034654307313407.

[29] Roediger, H. L., Karpicke, J. D., "Test-Enhanced Learning: Taking Memory Tests Improves Long-Term Retention," *Psychological Science*, vol. 17, no. 4, pp. 249–255, 2006, doi:10.1111/j.1467-9280.2006.01693.x.

[30] Dunlosky, J., et al., "Improving Students' Learning with Effective Learning Techniques: Promising Directions from Cognitive and Educational Psychology," *Psychological Science in the Public Interest*, vol. 14, no. 1, pp. 4–58, 2013, doi:10.1177/1529100612453266.

[31] Kasneci, E., et al., "ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education," *Learning and Individual Differences*, vol. 103, Art. 102274, 2023, doi:10.1016/j.lindif.2023.102274.